

</> fn main()

Proyecto personal · Sistemas de bajo nivel

Construir un JDK desde cero en Rust

Roadmap técnico por hitos

Autor: **Esteban Nicolás Larena**

Fecha: **14 de agosto de 2026**

Contenido

Introducción y alcance	3
Los tres pilares	3
El problema del bootstrap	3
Orden de construcción	4
Fase A — La JVM (el runtime)	5
Concurrencia — ruta al multithread profesional	7
Fase B — El compilador (javac, en Rust)	9
Compilador — pasadas y estructuras internas	11
Fase C — Las bibliotecas	12
Fase D — Herramientas (la “DK”)	13
Fase E — Cerrar el círculo	13
Más allá del JDK — la plataforma	15
Fase F — burst (optimizador)	15
Fase G — plain_data / value types	16
Fase H — KajiLibrary (biblioteca dogfooded)	17
Estado actual	18

Introducción y alcance

Este documento traza la ruta para construir un **JDK educativo desde cero en Rust**: no para competir con HotSpot, sino para **cargar y ejecutar bytecode real** y, eventualmente, compilar y correr código Java escrito y compilado con herramientas propias. El objetivo es doble: aprender sistemas de bajo nivel a fondo y completar un reto de ingeniería de gran calibre.

Un JDK no es una sola pieza. Son **tres pilares** que se sostienen entre sí:

Los tres pilares

Pilar	Qué hace	Lenguaje
JVM (el runtime)	Carga y ejecuta bytecode. Parser de <code>.class</code> → intérprete → objetos → heap → GC.	Rust
Compilador (<code>javac</code>)	Convierte <code>.java</code> en bytecode <code>.class</code> : lexer → AST → análisis semántico → emisión.	Rust
Bibliotecas (<code>java.base</code>)	<code>Object</code> , <code>String</code> , <code>System</code> , colecciones... escritas <i>en Java</i> , compiladas por el compilador propio.	Java

El problema del bootstrap

Las bibliotecas necesitan al **compilador** (para compilarse) y a la **JVM** (para ejecutarse). Si el compilador se escribiera en Java, se necesitaría a sí mismo para compilarse — el clásico problema del huevo y la gallina.

*Decisión de diseño: el compilador se escribe **en Rust**, igual que la JVM. Así Rust compila al compilador, el compilador compila las bibliotecas, y la JVM las ejecuta. El ciclo queda roto.*

Orden de construcción

La **JVM va primero**, sin excepción: tanto la salida del compilador como las bibliotecas son inertes sin un motor que las ejecute, y no se puede ni *probar* que el compilador genera bytecode correcto sin algo que lo corra.

A · JVM → B · Compilador → C · Bibliotecas → E · Cerrar el círculo
(motor) (.java → .class) (en Java) (todo junto)
 └─ D · Herramientas se completan en el camino (javap, java, jar...)

Horizontes: Base el núcleo, por aquí empezamos Avanzado más duro, segunda pasada Cumbre lo más difícil, pero vamos a llegar

*Cómo leer el roadmap: es una **ruta ordenada por hitos**. Cada hito tiene un criterio de éxito medible y no se cierra hasta cumplirlo. El orden respeta las dependencias: no se puede el hito N sin el N-1.*

Fase A — La JVM (el motor que ejecuta bytecode)

A0 Parsear el .class (≡ javap) núcleo logrado Base

Base Lector de bytes (cursor big-endian) — el `Reader` (`u1` / `u2` / `u4`)

Base Constant pool (17 clases de entrada; 1-indexed, Long/Double = 2 slots)

Base Header: magic, versiones, flags, this/super/interfaces

Base fields, methods, attributes: `Code`, `LineNumberTable`, `SourceFile`, `StackMapTable` (los 7 frame types)

Base Desensamblado de bytecode (tabla de opcodes completa) con comentarios `// ...` resueltos

Base Volcado `javap` brief y `-v` byte-idéntico; flags de visibilidad (`-public` / `-protected` / `-package` / `-p`)

Avanzado *Pendiente (no esencial):* `Signature`, `BootstrapMethods`, `InnerClasses`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `NestHost/Members`, `LocalVariableTable`

✓ **Éxito: logrado** — el volcado coincide **byte a byte** con `javap -v` (y brief) sobre 12 fixtures.

A1 Intérprete mínimo logrado Base

Base *Frame*: pila de operandos + variables locales

Base Contador de programa (PC) y *loop* de despacho de opcodes

Base Opcodes: `iconst`, `iload`, `istore`, `iadd`, `ireturn`

Base Parseo de descriptores de método (`(II)I`)

✓ **Éxito: logrado** — ejecutar un método que suma dos enteros (`Add.add`).

A2 Control de flujo y métodos logrado Base

Base Saltos: `if_icmpgt`, `goto`, comparaciones

Base *Method area* (metadatos de clases cargadas)

Base `invokestatic` + pila de frames (llamadas anidadas)

✓ **Éxito: logrado** — corren `factorial` recursivo (120) y `fibonacci` (55).

A3 Objetos y heap logrado Base

Base Heap + allocator simple; arena de bytes (`Vec<u8>`) + `malloc` bump

Base *Layout* de objetos (campos, con herencia) y de clases (con *vtable*)

Base `new`, `getfield` / `putfield`

Base `invokevirtual` / `invokespecial` / `invokeinterface` + dispatch dinámico (*vtable* + *itable*)

Base Arrays (objetos y primitivos int-category, con ancho fiel)

✓ **Éxito: logrado** — herencia (`Dog extends Animal`) y dispatch dinámico verificados: `a.sound()` sobre una referencia `Animal / Speaker` corre `Dog.sound`.

A4 Robustez del runtime logrado Base

Base Excepciones: `athrow` + tablas de excepción + *stack unwinding*; `finally`; excepciones **implícitas** (NPE, índice, cast, tamaño negativo)

Base *Linking*: resolución bajo demanda + **verificador** (type-checking con *StackMapTable*); errores de linkage (`NoClassDefFound` / `NoSuchMethod`)

Base Inicialización de clase (`<<clinit>`), perezosa, super-primero)

Base Jerarquía de class loaders (bootstrap/app + delegación)

✓ **Éxito: logrado** — un `try` / `catch` atrapa una `Boom` (con *unwinding* entre frames y `finally`), y `Base` / `Derived` inicializan en orden super-primero.

A5 Nativos + GC logrado Avanzado

Avanzado Puente a métodos nativos (I/O real) — `System.out.println` imprime de verdad

Avanzado *Intrinsics* — `getClass`, `hashCode`, `Math`, `arraycopy`, `String` / `Class`...

Avanzado GC mark & sweep — y más: **compactante**, free list con *coalescing*, política de fragmentación, 4 disparadores sobre *safepoint*, y **marcado transitivo** correcto

✓ **Éxito: logrado** — `System.out.println` imprime de verdad y el GC recolecta (y **compacta**) basura.

A6 Cumbre del runtime 🚧 en progreso Cumbre

Cumbre ✓ **Verificador de bytecode JVMs-estricto** — verifica con o sin `StackMapTable` (inferencia por punto fijo como fallback, y *cross-check* de la tabla contra la inferencia); gate **estructural** (§4.9: targets, no caerse del final, `jsr` / `ret`, tabla de excepciones); tipos de locales estrictos + cota de `max_stack`; reglas de **objetos sin inicializar** (`<init>` una sola vez); **handlers de excepción** tipados; y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`)

Cumbre ✓ **Sistema de tipos completo** — `int` / `long` / `double` / `float` ejecutados y verificados: cómputo, conversiones, comparaciones, división con excepción, categoría-2 (params/campos/estáticos/arrays/frames), y el **lattice de referencias** (covarianza de arrays + `join` / LUB)

Cumbre ✓ **GC generacional** — young (Eden + 2 *survivors*) con colector por **copia** (minor: evacúa/promueve por edad, estilo Cheney con *forwarding*), **write barrier + remembered set** para las raíces `old→young`, y Old con **mark-sweep-compact**; *minor* disparado al llenarse Eden, *major*/full por occupancy o explícito

Cumbre ✓ **Referencias débiles** (`java.lang.ref`) — `WeakReference` + `ReferenceQueue`: el major trata `referent` como débil y, al morir el referente, lo limpia (`get()→null`) y **encola** la referencia. Base para `PhantomReference` / `Cleaner` (post-mortem)

Cumbre **Hilos / concurrencia** (*en progreso*) — objetivo: **multithread profesional**, por fases. ✓ **Fase 0 — green threads**: `java.lang.Thread` + `start` / `join` / `sleep`, scheduler cooperativo round-robin en el safepoint, varios call stacks por-thread, GC sobre las raíces de **todos** los threads, terminación; verificado (`Threads.run → 100`) y visible en el step-visualizer. ✓ **Monitores**: `synchronized` de **bloques y métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma/suelta el monitor al entrar/salir del frame), reentrancia, señalización `wait` / `notify` / `notifyAll` sobre *wait-set/blocked-set*, `IllegalMonitorStateException`, `wait(timeout)` y monitor **GC-safe** (las claves se remapean por el *forward* del GC, así que se compacta con monitores tomados). ✓ **Substrato de SO + GIL (E1+E2)**: `JVM_THREADS=os-gil` corre cada `Thread.start()` en un `std::thread` real tras un **GIL** (`Arc<Mutex<SharedVm>>`, un opcode por lock), con `park` / `unpark` reales y `sleep` en tiempo real — correcto pero **serializado** (referencia/oráculo). ✓ **API de Thread (H1)**: `currentThread` / `yield` / nombre/id / `isAlive`, `getState()` con los seis estados de `Thread.State`, `new Thread(runnable)`, `start` dos veces → `IllegalThreadStateException`, e `interrupt()` que despierta `sleep` / `join` / `wait`. ✓ **Sacar el GIL (E3/H3 — paralelismo real, subconjunto conservador)**: `JVM_THREADS=os` corre los opcodes **frame-local** (aritmética int/stack/ramas) **lock-free en paralelo** sobre un `RunningCtx` por-hilo con code cache; solo `SharedVm` se lockea; el GC que mueve objetos frena a todos con un **safepoint stop-the-world cooperativo**. El `Arc<Mutex<JVM>>` de toda-la-VM desapareció. ✓ **Ensanchado**: (W1) frame-local completo (int/long/float/double, shifts, conversiones, refs, ramas); (W3) lecturas compartidas concurrentes bajo `RwLock` (`getField` / `arraylength` / array-loads); (W2) **asignación lock-free** (`new` / `newarray`, Eden en un arena `UnsafeCell` **verificado con Miri**). ✓ **JMM (H4 — modelo de memoria relajado)**: campos no-volatile **lock-free** (`Relaxed` atómico), `volatile` con `Acquire` / `Release` (`AtomicU64` sin *tearing* para `long` / `double`), Eden sobre un substrato atómico 8-alineado **Miri-verificado**; test de publicación `volatile` que coincide `green=os-gil=os`. ✓ **CAS (H5)**: `compareAndSwap` intrínseco (`cas_u32` / `cas_u64` sobre el `AtomicRegion`) + `AtomicInteger` / `AtomicLong` / `AtomicReference` (la CAS de referencia pasa por el write barrier). ✓ **Núcleo + primeras utilidades de java.util.concurrent** (H6): **AQS** (`AbstractQueuedSynchronizer`) en modo **exclusivo y compartido**, con `ReentrantLock` / `Condition`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `ArrayBlockingQueue` / `LinkedBlockingQueue` / `PriorityBlockingQueue` / `DelayQueue`, el **framework Executor** (`ThreadPoolExecutor` + `ScheduledThreadPoolExecutor` + `submit` / `Future` / `FutureTask` / `Callable`), un `ConcurrentHashMap` (encadenamiento + *lock striping*) y un `CompletableFuture` (`supplyAsync` / `thenApply` / `thenCompose`, completación excepcional con `exceptionally`, y `thenCombine` que fusiona dos futuros con un `BiFunction`) —todo en Java sobre AQS (o el monitor), validado `green=os-gil=os`; motivaron `Object.equals`, `java.util.function` / `Objects`, `Comparator` / `Comparable` / `Delayed` y `System.nanoTime`. ✓ **Periféricos de Thread**: `ThreadLocal` (aislamiento por-hilo real — lista de asociación colgada del `Thread` actual, keyed por identidad GC-safe; verificado con 4 hilos OS), **prioridad** (`get/set` + rango `[1,10]`) y **daemon** (atributo + regla de ciclo de vida). **Falta: el volumen** restante de j.u.c (dequeas, *resize*/lecturas lock-free del `ConcurrentHashMap`), locks por-estructura finos y `UncaughtExceptionHandler`; y un **bug residual** conocido — una referencia *stale* puede alcanzar un frame bajo **GC intenso concurrente** en `JVM_THREADS=os` (guard parcial landeado; próximo paso `ThreadSanitizer`; `green` / `os-gil`, el oráculo, **no** afectados). El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA), no en el heap Java. La hoja de ruta completa (JMM, CAS, `java.util.concurrent`) tiene su propia sección («Concurrencia»).

Cumbre ✓ **Cobertura del set de opcodes — 199/199 alcanzables: completo** — cerrados `nop` (0x00), `goto_w` (0xc8), el prefijo `wide` (0xc4: índice de local de 16 bits, la única forma de direccionar los slots pasados el 255; `wide inc` mide 6 bytes, el resto 4 — salió barato porque los handlers de locales ya reciben `slot: usize` y nunca se enteran del ancho, así que ensanchar es puro *decoding*) y `multianewarray` (0xc5: aloca recursivamente **sólo** los `dimensions` niveles indicados — `new int[3][ ]` deja los internos en `null` —, validando todos los conteos *antes* de alocar nada; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento, y las referencias hijo pasan por `store_reference` para que el *remembered set* vea los punteros `old→young`. Hubo que **modelarlo también en el verificador**, sin lo cual la clase corría con el verificador claudicando en silencio). Los 3 de `jsr` / `ret` / `jsr_w` quedan **excluidos por diseño** —JVMs §4.9.1 los prohíbe en class files de versión 50.0+ (Java 6 en adelante) — con la postura **leer sí, ejecutar no**: el desensamblador los soporta completo (requisito de A0, que se mide contra `javap` sobre `java.base`) y el gate estructural del verificador los rechaza. Nota de diseño: `subrutinas-jsr-ret.md`. El número honesto es entonces **199 de 199 alcanzables: completo**

Cumbre ✓ **invokedynamic** (0xba) — **no era un opcode, era un subsistema**, y corre: **5 de las 6 fábricas** que emite `javac`. `StringConcatFactory` (todo "a" + b desde Java 9), `SwitchBootstraps.typeSwitch` (`switch` sobre patrones de tipo y de enum), `ObjectMethods` (`equals` / `hashCode` / `toString` de un `record`, los tres desde una sola entrada de `BootstrapMethods`, distinguidos por el nombre del call site), `LambdaMetafactory.metafactory` (lambdas y method references) y `ConstantBootstraps.invoke` (constantes dinámicas). La sexta, `altMetafactory`, necesita serialización y queda **fuera de alcance**. `LambdaMetafactory` **genera una clase real** al vuelo (con el escritor de `.class`): implementa la interfaz funcional, con campos para las capturas y un método SAM que reenvía al handle — igual que el JDK (que spinnea con ASM), ya sin el atajo del objeto sintético. Y con el **modelo de objetos de java.lang.invoke** cerrado (abajo), `ConstantBootstraps.invoke` **se movió a Java** (es sólo `handle.invokeWithArguments(args)`); los otros *bootstrap methods* siguen como intrínsecos, con `java.lang.constant` en Java. Ruta completa, correcciones y mediciones: `invokedynamic-medic-ruta.md`

Avanzado ✓ **ldc de literales de clase** (`Foo.class`, `int[].class`) — empuja el mirror `Class<...>`, cacheado por Class ID (así `Foo.class == Foo.class`) y **sin inicializar** la clase: un literal no es *uso activo* (§5.5). Los de array reusan el mirror sintético que arma `anewarray`, ya que una clase de array no tiene `.class` que cargar

Cumbre ✓ **La VM puede invocar Java** (`call_java`) — empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el **caso general de lo que la inicialización de clases ya hacía**: `<clinit>` era la variante sin argumentos ni resultado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver tiene que reimplementar lo que necesite de la biblioteca. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals` / `hashCode` de sus componentes, y un `condy` ejecuta su bootstrap

Cumbre ✓ **Modelo de objetos de** `java.lang.invoke` (`MethodHandle` / `MethodType` / `Lookup`) — **cierra 0xba**. Las clases viven en **Java** (`bootstrap/java/lang/invoke/`), con `MethodHandle.invoke` / `invokeWithArguments` **nativos** — polimorfía de firma (§2.9.3), interceptados en `invokevirtual` antes de la resolución normal: el único intrínseco que de verdad corresponde. **El premio se cobró:** `ConstantBootstraps.invoke` es ahora **Java** (`return handle.invokeWithArguments(args)`), lo que exigió hacer el **cache de condy raíz de GC** (se remapea con el `forward` del colector). `ldc` de `MethodHandle` / `MethodType` —que `javac` **nunca** emite— se prueba con **class files hechos a mano por el escritor de** `.class` (su estreno). Kinds: `static/virtual/special/constructor` + **mirrors de primitivos** (`int.class` → `Integer.TYPE`, sin boxing). Y **LambdaMetafactory** **genera clases reales** al vuelo (vía el escritor de `.class`), implementando la interfaz funcional y reenviando al `handle` — como el JDK, ya sin el atajo del objeto sintético

Cumbre JIT (bytecode → código nativo)

✓ **Éxito: parcial** — **verificador JVMs-estricto, GC generacional**, el **set de opcodes completo** (`invokedynamic` incluido) e **hilos de SO con paralelismo real ensanchado** (GIL removido; cómputo + lecturas + asignación lock-free en `JVM_THREADS=os`, con el `unsafe` Miri-verificado) y el **JMM relajado** (campos no-volatile lock-free con `Relaxed`, `volatile` con `Acquire` / `Release`, Eden atómico) logrados; falta locks por-estructura finos y/o el JIT.

A7 Conformidad JVMs ✓ auditoría 2026-08-12 — 13/13 cerrados **Cumbre**

Cumbre ✓ **Default methods de interfaces (JSR 335, Java 8)** — estaba **roto** (`NoSuchMethodError`: la vtable heredaba sólo de la superclase). Ahora fusiona los defaults de las superinterfaces (BFS desde las directas = regla *maximally-specific* para todo lo que emite `javac`; la clase siempre gana; un método abstracto —sin `Code`— nunca toma slot). Cubre `invokeinterface` e `invokevirtual`. Los static de interface ya funcionaban. Test → 115 (`green=os-gil=os`)

Cumbre ✓ **Throwable completo** — `getMessage` / `toString` (nativo: lee el nombre de clase runtime) + **stack traces**: la VM captura la traza (`\tat pkg.Class.method`, innermost-first) en el `unwind` —punto único de faults implícitos y `throw`— y la guarda en el objeto (interning GC-safe); `printStackTrace()` la imprime. Constructores (`String`) en toda la jerarquía

Cumbre ✓ **ExceptionInInitializerError** (JVMs §5.5) — un `<clinit>` que lanza **propagaba... nada**: se tragaba el fallo y el `getstatic` devolvía 0. Ahora propaga al código que disparó la init (un `floor` de unwinding por `call_java` + `pending_exception`), envuelve si no es `Error`, y deja la clase **erroneous** → `NoClassDefFoundError` al reuso

Cumbre ✓ **Uncaught exception** (§2.10) — estaba **roto**: una excepción sin catch hacía `panic!` de **toda la VM**. Ahora imprime el formato exacto del JDK (`Exception in thread "Thread-1" java.lang.RuntimeException: ...` + la traza capturada en el `throw`) y termina **solo ese thread** — los joiners despiertan, `main` sigue; si es `main`, termina el programa. Test → 42 (`green=os-gil=os`). Falta la API `setUncaughtExceptionHandler`

Cumbre ✓ **ArrayStoreException** (§6.5) — `aastore` hace el chequeo dinámico (clase runtime del valor vs tipo de elemento, `is_subtype` con covarianza; null siempre válido). Test → 42. **Bonus: 2 bugs latentes expuestos y arreglados** — los mirrors de clases array se alocaban en **Eden** (el minor GC los movía → índice/headers colgados → ASE espurio en stores válidos; ahora **Old-pinned** como todo mirror) y `anewarray` nombraba `[LJ`; en vez de `[J` (intérprete + verificador). Regresión → 64

Cumbre ✓ **StackOverflowError** (§6.3) — `MAX_FRAMES = 2000` en `push_frame_locked` (el embudo de los 5 invokes), chequeado **antes** de adquirir el monitor (no filtra locks de `synchronized`); `call_java` cubierto vía `pending_exception`. Test → 42

Cumbre ✓ **OutOfMemoryError** (§6.3) — agotamiento **recuperable para las alocaiones de bytecode** (`new / newarray / anewarray / multianewarray` → `try_malloc` → `throw`); las internas del VM conservan el `panic`, documentado. Test → 42 (recursión roteando 512 KiB/frame agota los 16 MiB reales)

Cumbre ✓ **Autoboxing / wrappers** (JLS §5.1.7) — los 8 wrappers reales: `Integer` / `Long` / `Short` / `Byte` con `valueOf` + **cache -128..127** (identidad `==` verificada: `100==100 true` / `200==200 false`), `Character` (0..127), `Boolean` (TRUE/FALSE canónicos), `Double` / `Float`. Test → 42. Colateral: destapó un **2º reproducir del heisenbug os-parallel** (single-thread ~50%, `Long.<clinit>` + Eden lleno → `ArithmeticException` espuria; `java/BxDbg{T,Y}.java`)

Cumbre ✓ **Object.clone()** + **Cloneable** (JLS §10.7) — copia shallow interceptada en `invokevirtual` e `invokespecial` (`super.clone()`), `Cloneable` → `CloneNotSupportedException`; **arrays incluidos** (`"I".clone()` pre-vtable). Clon alocado en **Old** (una alocación Eden podría mover el fuente a mitad de copia); refs vía write barrier. Test → 42

Cumbre ✓ **Class.getName()** / **getSimpleName()** — nativos sobre el mirror, formato JDK completo (clases con puntos; arrays `[Ljava.lang.String;` / `String[]` / `int[]`). Test → 42 (incl. `String.class.getName()` vía `ldc` de `Class`)

Base ✓ **Anotaciones en runtime** (JSR 175) — `Class.isAnnotationPresent(Class)` sobre el `RuntimeVisibleAnnotations` ya parseado. **Fuera de alcance:** `getAnnotation()` devolvería un **objeto** anotación → hacen falta clases proxy por `@interface` (dynamic proxies + modelo de `Method`)

Base ✓ **java.lang.ref: Soft + PhantomReference** — phantom con `get()` → null pero `referent` intacto (el GC encola igual; el override impide resucitar). La política soft vive en el **GC**: el `referent` se traza **fuerte** salvo en una colecta pedida por **presión** de memoria, donde muere como un weak. Verificado en ambas direcciones (42 normal / 36 bajo presión). **Fuera de alcance:** `Cleaner`; `finalize()` no se implementa por decisión

Base ✓ **Ciclo de vida: System.exit + Runtime** — interceptado en `invokestatic` (el bridge de nativos solo sabe *continuar*; terminar exige un `Step`): descarta los frames **sin** liberar monitores ni desenrollar —el apagado ordenado que `exit` no debe hacer— y propaga el status desde cualquier thread. Contrastado con el `java` real. **Fuera de alcance:** shutdown hooks

Base ✓ **Periféricos de Thread** — **UncaughtExceptionHandler invocado de verdad en Java**, y **antes** de pepear el último frame: el lookup aloca y ese frame es la única raíz de GC que sostiene la excepción; un handler que lanza queda contenido por el `exception_floor`. Más `sleep(long,int)`, `onSpinWait()` y `ThreadGroup` (registro enteramente en Java)

Base ✓ **Regla fina de init de interfaces** (§5.5) — **era un bug**: el `<clinit>` de una interface con default methods nunca corría (ni directo ni indirecto). Ahora un BFS junta las superinterfaces que declaran un método con `Code`, y **devuelve vacío si la clase es una interface** — la regla "una interface no inicializa sus superinterfaces" queda en el tipo de retorno, no en un `if`

Base ✓ **Cubierto ya auditado:** class file 199/199, verificador, JMM, indy 5/6, j.u.c núcleo, records/patterns. **Fuera de alcance por diseño:** módulos (JSR 376) y NIO en la VM, `finalize()`, y lo que depende de reflexión completa (Fase C)

✓ **Éxito: auditoría CERRADA, 13/13** (2026-08-12) — el runtime contra la JVM Specification y los JSR estructurales (detalle: `holdon.md`). Los 8 hallazgos gruesos (default methods, uncaught exceptions, `ArrayStoreException`, `StackOverflowError`, `OutOfMemoryError`, autoboxing, `clone`, `Class.getName()`) y los 5 menores (anotaciones runtime, `Soft/Phantom refs`, `System.exit` / `Runtime`, periféricos de `Thread`, init de interfaces), **cada uno validado por el oráculo** (`green=os-gil=os` + 10× os-parallel). Tres resultaron **bugs reales** que nadie había visto: los mirrors de clases array vivían en Eden, `anewarray` nombraba mal los arrays anidados, y el `<clinit>` de una interface con default methods no corría nunca. De paso apareció un 2º reproducir del heisenbug os-parallel. Lo único que queda es lo marcado **fuera de alcance** con su razón (`getAnnotation` sin dynamic proxies, `Cleaner`, shutdown hooks, `finalize()`).

Concurrencia — ruta al multithread profesional

La ejecución concurrente (Hito A6) merece su propio mapa, porque es donde el proyecto apunta a **multithread profesional**. Lo que hoy corre sobre **green threads** es la *planta baja* de un edificio cuya cumbre es `java.util.concurrent`. Cada planta se apoya en la de abajo: no hay `ReentrantLock` sin CAS, ni CAS útil sin un modelo de memoria, ni modelo de memoria que *importe* sin hilos reales que lo hagan necesario.

RASCACIELOS DE LA CONCURRENCIA · construir hacia arriba ↑

[6] java.util.concurrent · núcleo + utils · falta vol. HECHO

[x] AQS (excl + shared) · ReentrantLock/RWLock · Condition

[x] Semaphore · CountDownLatch · CyclicBarrier · ArrayBlockingQueue

[x] ThreadPoolExecutor · ScheduledThreadPoolExecutor · Future

[x] ConcurrentHashMap · Linked/Priority/DelayQueue · Completabl

[] deque · resize/lecturas lock-free CHM · locks finos

[5] Atómicos / CAS · raíz lock-free HECHO

[x] compareAndSwap · AtomicInteger / AtomicLong / Reference

[4] Modelo de Memoria (JMM) · relajado (H4)

✓

●

[x] volatile=Acq/Rel · no-volatile Relaxed · no-tearing · Eden atómic

[3] API de Thread · control de hilos HECHO

[x] start · run · join · sleep · currentThread · yield

[x] getState (6 estados) · isAlive · name/id · start x2

[x] interrupt despierta sleep/join/wait · daemon/prio/TLocal

[2] Monitor intrínseco · monitor completo HECHO

[x] monitorenter/exit · synchronized (bloques Y métodos)

[x] reentrancia · wait / notify / notifyAll · IMSE

[x] wait(timeout) · GC-safe · interrupt-of-wait

[] biased/thin locks · deadlock detect

[1] Substrato de ejecución · paralelismo real

✓

●

[x] green threads (cooperativo, default + visor)

[x] hilos de SO reales · park/unpark · os-gil (referencia)

[x] GIL removido: frame-local + lecturas + alloc lock-free

[x] W2 alloc: Eden en arena UnsafeCell (Miri-verificado)

[x] JMM (H4) relajado · [] locks por-estructura finos

▲ todo descansa en la planta 1 ▲

Estado:

✓

 hecho

●

 parcial

■

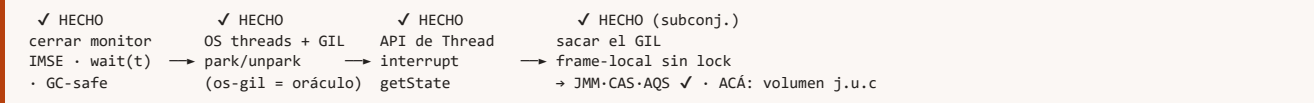
 falta

Planta	Estado	Qué falta para “profesional”
6 · <code>java.util.concurrent</code>	🟢🟡 núcleo + utils (H6)	Hecho: AQS (<code>AbstractQueuedSynchronizer</code>) en modo exclusivo y compartido —base de casi todo—, <code>ReentrantLock</code> + <code>Condition</code> , <code>ReentrantReadWriteLock</code> , <code>Semaphore</code> , <code>CountDownLatch</code> , <code>CyclicBarrier</code> , <code>ArrayBlockingQueue</code> / <code>LinkedBlockingQueue</code> / <code>PriorityBlockingQueue</code> / <code>DelayQueue</code> (una/dos locks / min-heap / delay), el framework Executor (<code>ThreadPoolExecutor</code> + <code>submit</code> / <code>Future</code> / <code>FutureTask</code> / <code>Callable</code> , y un <code>ScheduledThreadPoolExecutor</code> con min-heap por deadline), un ConcurrentHashMap (encadenamiento + <i>lock striping</i>) y un CompletableFuture (<code>supplyAsync</code> / <code>thenApply</code> / <code>thenCompose</code> / <code>get</code> , <code>exceptionally</code> y <code>thenCombine</code>), todo en Java sobre AQS (o el monitor) y validado por el oráculo (<code>green==os-gil==os</code>). Motivaron agregar <code>Object.equals</code> , <code>java.util.function</code> / <code>Objects</code> , <code>Comparator</code> / <code>Comparable</code> / <code>Delayed</code> y el intrínseco <code>System.nanoTime</code> . Falta (volumen): dequeues, <i>resize</i> /lecturas lock-free del <code>ConcurrentHashMap</code> .
5 · Atómicos / CAS	🟢 hecho (H5)	Hecho: <code>compareAndSwap</code> (la instrucción atómica raíz: <code>cas_u32</code> / <code>cas_u64</code> sobre el <code>AtomicRegion</code> , <code>AcqRel</code>) y <code>AtomicInteger</code> / <code>AtomicLong</code> / <code>AtomicReference</code> —la CAS de referencia pasa por el write barrier—. Verificado: <code>AtomicLong</code> 64-bit + <code>AtomicReference</code> identity-CAS → 202.
4 · Modelo de Memoria (JMM)	🟢🟡 relajado	Hecho (H4): campos no-volatile lock-free (<code>Relaxed</code>), <code>volatile</code> = <code>Acquire</code> / <code>Release</code> , no-tearing de <code>long</code> / <code>double</code> (<code>AtomicU64</code>), Eden atómico Miri-verificado ; el cache de condy es raíz de GC. <i>happens-before</i> vía los atómicos + el <code>RwLock</code> . Falta: semántica de <code>final</code> , <code>VarHandle</code> / CAS.
3 · API de <code>Thread</code>	🟢 hecho (H1)	Hecho: <code>start</code> / <code>run</code> / <code>join</code> / <code>sleep</code> , <code>currentThread</code> , <code>yield</code> , nombre/id, <code>isAlive</code> , <code>getState()</code> (los seis estados; <code>ThreadStatus</code> pasó a cinco + <code>NEW</code> derivado, con <code>sleep_until</code> / <code>joining_on</code> plegados dentro), <code>Thread(Runnable)</code> , <code>start</code> dos veces → <code>IllegalThreadStateException</code> , e <code>interrupt</code> / <code>InterruptedException</code> que despierta <code>sleep</code> / <code>join</code> / <code>wait</code> (re-adquiere el monitor antes de lanzar en el <code>wait</code> ; la carrera <i>notify/interrupt</i> la resuelve el GIL). Periféricos hechos: <code>ThreadLocal</code> (aislamiento por-hilo verificado), prioridad (rango <code>[1,10]</code>), daemon (regla de ciclo de vida). Falta: <code>UncaughtExceptionHandler</code> .
2 · Monitor intrínseco	🟢 casi	Hecho: <code>monitorenter</code> / <code>exit</code> , <code>synchronized</code> (bloques y métodos), reentrancia, <code>wait</code> / <code>notify</code> / <code>notifyAll</code> , <code>IllegalMonitorStateException</code> , <code>wait(timeout)</code> y monitor GC-safe (las claves se remapean por el <i>forward</i> del GC en minor/compact, así que se puede compactar con monitores tomados). Falta: interrupción del <code>wait</code> (llega con H1), biased/thin locks, detección de deadlock.
1 · Substrato de ejecución	🟢🟡 gran avance	Hecho (E1+E2): green threads (default y visor) y hilos de SO reales (<code>std::thread</code> por <code>Thread.start()</code>); <code>park</code> / <code>unpark</code> reales. Hecho (E3 — GIL removido): en <code>JVM_THREADS=os</code> los opcodes frame-local (aritmética int/stack/ramas) corren lock-free en paralelo sobre un <code>RunningCtx</code> por-hilo con code cache; solo <code>SharedVm</code> se lockea; GC con safepoint stop-the-world cooperativo . El <code>Arc<Mutex<JVM>></code> de toda-la-VM desapareció; <code>os-gil</code> queda como oráculo. Ensanchado: frame-local completo (W1), lecturas concurrentes bajo <code>RwLock</code> (W3), y asignación lock-free con Eden en un arena <code>UnsafeCell</code> Miri-verificado (W2). JMM (H4) hecho: modelo relajado — campos lock-free <code>Relaxed</code> , <code>volatile</code> <code>Acquire</code> / <code>Release</code> , Eden atómico. Falta: locks por-estructura finos, preempción.

Los dos cimientos reales

De todo el edificio, **dos primitivas** sostienen el resto: `park` / `unpark` (planta 1) y **CAS** (planta 5). De esas dos se deduce *casi todo* lo de arriba — AQS, los locks, los atómicos, los pools. Por eso el salto grande **no** era terminar el monitor (planta 2), sino cambiar el **substrato** de green threads a hilos de SO — y ese salto **ya está dado** (E1+E2): `park` / `unpark` son reales y el monitor quedó cerrado salvo la interrupción del `wait`. Queda la segunda mitad: **sacar el GIL** (E3), y después CAS como cimiento de todo lo lock-free.

Ruta crítica



El camino canónico — el mismo que recorrieron CPython y las JVM tempranas — es **GIL primero** (hilos de SO con un lock global que serializa el intérprete: simple y correcto, pero todavía sin paralelismo real) y después **sacar el GIL** (locks finos por estructura + TLABs + GC stop-the-world). Recién ahí `volatile` y los *fences* dejan de ser decorativos: con green threads no hay reordenamientos reales que tapar; con hilos de SO, omitirlos rompe el código de formas no-deterministas.

Para el LLM, el paralelismo pesado **no** vive en este rascacielos: vive en **kernels off-heap** (rayon/CUDA). Esta torre es para la **corrección** de la concurrencia Java, no para el cómputo numérico — que sale del heap por completo.

Fase B — El compilador (javac, escrito en Rust)

El front-end convierte texto en estructuras y la back-end las convierte en bytecode:

```
.java → lexer (tokens) → parser (AST) → análisis semántico → generación de bytecode → .class
```

B0 Lexer **logrado** Base

Base Scanner con **maximal munch**; comentarios; literales `int / long / float / double / char / String` (hex/bin/octal, **text blocks** `"""..."""` — el lexema crudo; el destripado §3.10.6 lo hace el parser)

Base `TokenKind` espejo **fiel** del enum de javac (115 constantes; las keywords contextuales `var / record / sealed...` van como identificador)

✓ **Éxito: logrado** — tokeniza el juego **completo** de Java (las 115 `TokenKind` de JDK 25).

B1 Parser **logrado (lenguaje completo, sin bordes)** Base

Base Gramática → AST. **Auditado contra el §14**: están **todas** las sentencias —incluida la **clase local** (§14.3)— bloque, declaración local (múltiple, `final`, y con declarador al estilo C `int y[]`), `;`, etiquetada, expresión, `if`, `assert`, `switch` (dos puntos y flecha, con patterns y guardas), `while`, `do`, `for` (básico/multi-init/infinito) y `for-each`, `break` / `continue` **con y sin etiqueta**, `return`, `throw`, `synchronized`, `try` / `catch` / `finally` con multi-catch y recursos, y `yield`. Más los **inicializadores** `static { }` y `{ }`, el **inicializador de array** `{1,2,3}` (§10.6) y el `instanceof` con **pattern**

Base *Precedence climbing* + *backtracking* (declaración local vs. expresión, `cast` vs. paréntesis, **cierre de genéricos** — parte `>> / >>>` con *undo log*)

Base **Genéricos**: argumentos de tipo (`List<String>`), *wildcards* (`? extends / ? super`), parámetros con cotas (`<T extends A & B>`), el **diamante** `<>`

Base **Visualizador de AST** (árbol ASCII) + volcado de tokens en el binario `javac ; y javac-step`: visor **paso a paso de las pasadas** (tokens → AST → tabla → chequeo)

Base  **Formas de expresión modernas** — cerradas esta iteración, todas parseadas y guardadas fielmente en el AST (compilarlas es de B2/B3, y hasta entonces la barrera del emisor las corta): **lambdas** (§15.27, con el desambiguador (`-cast-vs-paréntesis-vs-parámetros` por *lookahead* de la flecha), **referencias a método** (§15.13, incl. `T[]::new` por los corchetes vacíos), **clases anónimas** (§15.9.5, cuerpo de clase con nombre vacío), **clases locales** (§14.3) y el **type witness** (`Collections.<String>emptyList()`) —que además **se aplica**: fija los parámetros de tipo del método en vez de inferirlos, así un override deliberado incompatible ahora falla—

Base  **Text blocks** (§3.10.6) — el lexer toma el lexema crudo `"""..."""` y el parser **decodifica** el valor con el algoritmo del JLS **en orden**: (1) normaliza los terminadores de línea a `\n` (CRLF/CR), (2) descarta la **línea de apertura** (tras `"""` solo puede ir *white space* + un terminador), (3) quita la **indentación incidental** —el mínimo común de sangría de las líneas **no-blancas** y de la del **delimitador de cierre**— y los **blancos finales** de cada línea, y (4) **recién entonces** procesa los escapes, con los dos propios del text block: `\s` (un espacio que **sobrevive** al destripado de finales, porque este corre antes) y la **continuación de línea** `\<LF>` (una barra al final de línea se **traga** el salto). El resultado es un `String` común que el resto del pipeline emite como cualquier literal (`ldc`), verificado de punta a punta

Base  **Anotaciones** (§9.7) — antes se **descartaban** (`skip_annotation`); ahora `modifiers()` las lee junto a los modificadores y las **cuelga** de la declaración (clase, método, campo, parámetro). Se parsean enteras: marcador, valor único, pares con nombre, arreglos y anotaciones anidadas. Metadata, no bytecode: emitir el atributo `RuntimeVisibleAnnotations` es de B3. **Lección de la auditoría**: los inicializadores `static { }` / `{ }` también se *parseaban* y se *descartaban* —el código desaparecía sin que nada fallara—; hoy se conservan. *Parsear y tirar* es peor que no parsear, porque no deja rastro

Avanzado  **Bordes cerrados** — el parser ya no tiene huecos: las anotaciones sobre un **parámetro de tipo** (`<@Foo T>`) y sobre una **constante de enum** ahora se **retienen** (`TypeParam.annotations / EnumConstant.annotations`), las **anotaciones de tipo/uso** (§9.7.4, `List<NonNull String>`, `@Foo int`) se **aceptan** —se descartan por ser metadata ajena a la *erasure*— y el **type witness de constructor** (`new <T>Foo()`) ya **no da error**. Y se cerró la última: la anotación de **nivel de array** (`String @A []`, una por dimensión en `int @A [] @B []`) se **acepta y descarta** —tras la base, un `@` suelto solo puede ser de nivel de array—. **El parser queda sin bordes**

✓ **Éxito: logrado** — AST por *recursive descent* de clases/miembros, sentencias y expresiones con toda la precedencia; **genéricos, lambdas, referencias a método, clases anónimas y locales, type witness y anotaciones** incluidos.

B2 **Análisis semántico** **pasadas 1 y 2 + chequeos** **Base**

Base  **Pasada 1 — Enter/MemberEnter:** tabla de símbolos completa — paquetes, tipos **anidados**, `enum` / `record` / `@interface`, **genéricos con cotas**; **class finder real** (lee `.classpath` del classpath, incl. el atributo `Signature` → jerarquía genérica del JDK); resolución + validación con errores **posicionados**; síntesis implícita; y el **grafo tipado persistido** como salida

Base  **Pasada 2 — Attribute:** resuelve nombres + tipa **los cuerpos**, **decorando el AST in situ** (tipo por nodo, *binding* local/campo/método, slot de cada local con categoría-2, **variables de patrón** de un `case T v` con su slot, y el **constructor resuelto** de cada `new` para que el codegen sepa qué descriptor emitir) — la salida que consume el codegen

Avanzado  **Overload resolution en 3 fases** (JLS §15.12.2: estricta → *boxing* → *varargs*), con *most specific* y detección de ambigüedad

Avanzado  **Genéricos** (etapas A-B-C): subtipado con *containment* de wildcards (invariancia), *erasure*, sustitución del receptor (`List<String>.get` ⇒ `String`), `lub` / `glb`, e **inferencia** de los argumentos de tipo de una llamada (Cap. 18: reducción → incorporación → resolución)

Avanzado  **Errores a nivel de expresión** — cada error lleva la línea:col del **nodo**, no del método (`int b = a + noSuchVar`; apunta a `noSuchVar`, no a la sentencia); documentado en `docs/pasada2-attribute.md` y `docs/semantica-jdk25.md`.  **Indulgencia con externos acotada** — antes, un miembro no hallado en un tipo del classpath se aceptaba en silencio. La raíz eran dos: no se cargaba la **cadena de supertipos** (un método heredado, `s.hashCode()` de `Object`, podía no aparecer), y los nombres del `.class` venían con puntos donde el *finder* esperaba barras (la superclase no cargaba). Cerrados los dos: se cargan los supertipos **transitivamente**, y la indulgencia quedó **acotada** — un **nombre inexistente** (`s.noExiste()`, `ArrayList.noExiste()`) ahora se reporta si la jerarquía está **completa**; sigue indulgente solo la **sobrecarga que no matchea** (nuestra resolución no cubre toda firma del JDK) y las jerarquías **incompletas** (classpath del JDK ausente → red de seguridad). Con esto **B2 no tiene deudas de rigor conocidas**

Avanzado  **Chequeos semánticos** (esta iteración, en la pasada **Check** y en **Attribute**): **control de acceso** (§6.6 — `private` / `protected` / paquete sobre cada uso), **excepciones chequeadas** (§11.2 — toda `checked` que se lance tiene que estar capturada o en el `throws`; requirió **retener la cláusula** `throws`, que el parser descartaba, y guardarla en la tabla), **this / super en contexto estático** (§8.4.3.2), **reasignar un campo final** fuera de constructor/inicializador (§8.3.1.2) y el **acceso a tipo anidado cualificado** (`Outer.Inner.v()`, §6.5.5 — el único que *rechazaba código válido*). El chequeo de acceso y el de excepciones corren sobre el árbol **fuentes**, antes del desugar, para no tropezar con el código sintético (el constructor del `enum` llama a `Enum.<init>`, que es `protected`)

 **Éxito: logrado (núcleo)** — acepta un programa bien tipado y rechaza uno con error de tipos/nombres, con **overload resolution** y **genéricos** reales; semántica **completa** de JDK 25 como norte, citada a la JLS 25.

B3 Generación de bytecode **✓ núcleo + StackMapTable + invokedynamic** **Base**

Base **✓ Pipeline completo en** `compile()` — las cinco pasadas en el orden de javac: `Enter` → `Attribute` → `Flow` → `Desugar` → `(re-Attribute)` → `Codegen`.

Flow va antes que Desugar (sus errores apuntan al código fuente, no al bajado) y se **re-atribuye después** (las reescrituras dejan nodos sin decorar y el emisor necesita tipo/binding/slot). Un programa con errores semánticos **no emite** `.class`

Base **✓ Constant pool** con deduplicación + escritor de `.class` (v69) propio:

`Utf8 / Class / NameAndType / Methodref / Fieldref / Integer / String / Long / Double / Float`, con el hueco que exige la JVMs tras cada `Long / Double` (ocupan dos entradas); sección de **campos** y atributos `Code / SourceFile`

Base **✓ Tipos anchos y String** — `lconst / dconst / fconst`, `ldc2_w` para constantes de categoría 2, `ldc` para `String`, y la **promoción numérica binaria** (§5.6.2) con los 12 opcodes `x2y` (`longA + 1` inserta el `i2l`), con los desplazamientos como excepción (§15.19)

Base **✓ Objetos** — `new + dup + invokespecial <init>`, `getfield / putfield`, `getstatic / putstatic`, `invokevirtual` con receptor, `checkcast` y los estrechamientos `i2b / i2c / i2s`; **asignación** a local y a campo; y el `super()` implícito de todo constructor (§8.8.7), sin el cual el `this` queda sin inicializar y el verificador rechaza el `putfield`

Base **✓ Flujo de control** — etiquetas con parcheo de saltos; `if / else`, `while`, `do-while`, `for`, `break`, `continue`; comparaciones por categoría (`if_icmp`, `if_acmp`, `lcmp / fcmp1 / dcmp1 + if*`) y `&& / ||` compilados como **saltos** (cortocircuito real, no un booleano calculado)

Base **✓ Excepciones** — `try / catch` con su tabla, `throw`, y el `finally` **duplicado** en la salida normal y en un handler `catch-all` que re-lanza (§14.20.2): la v69 ya no acepta `jsr / ret`, así que duplicar es la única vía. (Esto reubicó el *inlining* del `finally`, que teníamos mal anotado como tarea del **desugar**: es del emisor)

Avanzado **✓ StackMapTable** (§4.7.4) — el frame de cada destino de salto, como el **merge** de los estados que llegan ahí (un slot que no coincide en todos los caminos pasa a `Top`). Siempre `full_frame`: legal en cualquier posición y evita las formas comprimidas. Los *handlers* llevan `stack = [E]` y hay que **forzarles** el frame, porque no los alcanza ningún salto. La pila de operandos se lleva **tipada**, no como simple altura: es lo que el frame tiene que declarar, y sin eso un destino alcanzado con algo ya empujado (`f(a, b > 0 ? 1 : 2)`) declaraba la pila vacía. Eso incluye el tipo `uninitialized` (tag 8), que identifica un objeto recién creado por el **offset de su** `new`; corrido su `<init>`, todas sus apariciones —pila y locales— pasan al tipo definitivo (§4.10.2.4). Validada con el *cross-check* del verificador propio

Base **✓ Barrera de seguridad** — `generate()` devuelve `Result`: una construcción no soportada **falla con posición** en vez de emitir un `.class` a medias. Destapó dos agujeros graves que el propio desugar venía alimentando en silencio — `instanceof` (que produce todo *pattern switch*) y la **creación de arrays** (que produce todo *varargs*)—, que a partir de ahí se cerraron. Si un `for-each / assert / yield` llega al emisor, el mensaje dice que es un **bug del desugar**, no una limitación

Base **✓ Arrays e instanceof** — `newarray / anewarray`, `arraylength`, las familias `Xaload / Xastore` (donde `byte / char / short` llevan **su propio** opcode aunque compartan la categoría `int`), inicializadores (`new int[] {4,5,6}`) y escritura de elementos. Con esto **compilan y corren** las llamadas *varargs* y los *pattern switch*, que el desugar ya venía generando

Base **✓ Ternario, switch, saltos etiquetados y synchronized** — el `?:` con sus dos ramas **promovidas al tipo del todo** (§15.25); el `switch` como `tableswitch` o `lookupswitch` según la **densidad** de las etiquetas (la heurística de javac), con el relleno de alineación y los desplazamientos de 4 bytes, preservando el *fall-through* de la forma de dos puntos y cortándolo en la de flecha; `break / continue` **con y sin etiqueta** sobre una pila de sentencias «de las que se puede salir» —un `switch` interpuesto captura un `break` pero **no un continue** (§14.16)— más el **bloque etiquetado**; y `synchronized` con `monitorexit` en **las dos** salidas, la normal y un handler `catch-all` que re-lanza, con el monitor copiado a un local por el desugar para no reevaluarlo. De yapa, el `switch` sobre `String` pasó a compilar de punta a punta: el desugar ya lo bajaba a dos switches sobre `int`, pero ninguno de los dos se podía emitir

Base **✓ Emisión multi-clase** — `generate()` emite un `.class` **por tipo** (top-level y anidados, recursivo), cada uno con **su** nombre (`Multi`, `Multi$Nested`, `Otra`, el `E$1` del `switch -enum`). Era la última fuga silenciosa: antes se escribía **una sola clase** con el nombre del archivo, y un segundo tipo se perdía sin aviso. De paso, el `switch` sobre `enum` **ahora corre** de punta a punta (su `$SwitchMap` vive en `E$1`, que recién ahora llega al disco)

Avanzado **✓ invokedynamic** (lambdas, *method refs* y *record*) — un nodo **Indy genérico** (bootstrap + argumentos estáticos **tipados**: `MethodType / MethodHandle / Class / String`) que el desugar produce y el emisor serializa sin recomputar. Las **lambdas** se bajan con un `LambdaToMethod` propio: método sintético `lambda$...` con las capturas de cabecera (análisis *effectively-final*, reusando el de las lambdas), `this` capturado cuando el cuerpo lo usa (impl de **instanciá**, `REF_invokeSpecial`; si no, estática, `REF_invokeStatic`), y el `invokedynamic` con `LambdaMetafactory.metafactory` de *bootstrap* (`MethodType` borrado/instanciado + el `MethodHandle` de la impl como argumentos estáticos). Las **referencias a método** cubren las **cuatro formas** del §15.13.1 (estático, instancia **ligado/no-ligado**, constructor) apuntando al método/constructor **real**, sin sintetizar nada. El `equals / hashCode / toString` de un `record` (§8.10.2) va por `ObjectMethods.bootstrap`, con la `Class`, la cadena de nombres de componentes y un `MethodHandle` `REF_getField` por componente. El pool ganó `MethodHandle / MethodType / InvokeDynamic` + el atributo `BootstrapMethods`, y el **verificador propio** el opcode `0xba`. La **ejecución** del call site vive en la JVM (KajiJDK); acá el oráculo es el verificador estricto

Avanzado **✓ Clases internas, locales y anónimas** (§8.1.3/§14.3/§15.9.5) — captura del entorno **completa**, las tres formas **corren y verifican**. Una **interna de instancia** captura la instancia envolvente en un campo `final Outer this$0` (parámetro de cabecera + asignación en el constructor, `new Inner() → new Inner(this)`, y el acceso sin cualificar a un miembro del enclosing reescrito a `this$0.f / this$0.m()`, con la resolución por la cadena de `owner` sumada a `Attribute`). Una **local** (registrada por la pasada mutable `register_local_classes` —un *body-walk* que la renombra a única y le da binary `Outer$1L` —, tipada *in situ* con los locales del método visibles) captura además los locales *effectively-final* en campos `val$x` —parámetros de constructor que `new L(caps)` pasa— reusando el análisis de las lambdas; y combina `this$0 + val$` en método de instancia. Una **anónima** (`new Base(){...}` o `new Runnable(){...}`) se baja a una **local sintética** `Outer$N` en un pase `hoist_anonymous` entre `Enter` y `Attribute`: recorre el mismo camino que la local (registro, tipado, levantado, captura), **extiende una clase o implementa una interfaz** (según el `ty` resuelto) con override y captura `val$ / this$0` — las dos corren y verifican. **✓ Argumentos de super** (`new ArrayList(10){...}`): la anónima sintetiza en `$N` un constructor que los **reenvía** a `super(...)`. **✓ Outer.this** (§15.8.4): baja a la cadena de `this$0 (this.this$0...)`, desambiguando un campo **sombreado** (`Outer.this.f` vs. `this.f` van a clases distintas) y resolviendo el **anidamiento multinivel** subiendo por la cadena de `owner`. **✓ outer.new Inner()** (§15.9.2): el **calificador** se empuja como `this$0` en vez de `this` —válido incluso en contexto **estático**, donde no hay `this`—. **✓ Colisión de nombres** (§14.3): dos locales homónimas (`L` en dos bloques del mismo enclosing) ya no colisionan —antes la tabla clavea el tipo por nombre y la 2ª se **descartaba en silencio**—. Una pasada mutable **renombra** cada local a un nombre único por unidad (`L → 1L, 2L ...`, binarios `Outer$1L / Outer$2L`) y **reescribe sus referencias con alcance léxico por bloque** (visible de la declaración al fin del bloque), dejando al resto del pipeline ver nombres únicos en un scope plano. Destapó de paso un bug del desugar —`has_this` no

se restauraba al procesar el cuerpo de una local, y una segunda local en un método **estático** creía capturar `this$0` —, ya cerrado.  **Local-en-local** (§14.3): una local declarada dentro del método de otra local se registra como **tipo anidado suyo** (`Outer$1L1$2L2`) —la pasada de registro recorre el cuerpo de una local tomándola a ella como nuevo enclosing— y el desugar la baja recursivamente.  **Multinivel implícito** (§8.1.3): el acceso **sin cualificar** a un miembro de un enclosing lejano (una interna dos o más niveles adentro que lee un campo/llama un método del abuelo) encadena tantos `this$0` como niveles (`this.this$0.this$0.f`), construyendo la cadena de enclosings por `owner`. Con esto las clases internas quedan **sin colas**

Avanzado  **Emisión de interfaces propias** — el `.class` de una `interface` sale con `ACC_INTERFACE`, la lista de super-interfaces y sus métodos **abstractos sin Code** (§4.6); el emisor ya no sintetiza constructor para ella. Se sumó `invokeinterface` (con `InterfaceMethodref`, tag 11, y su `count`) para las llamadas sobre un receptor de interfaz —antes se emitía `invokevirtual`, mal, así que también arregló las llamadas a `List / Comparator` —, y el *method ref* a método de interfaz (`REF_invokeInterface`) pasó a `InterfaceMethodref`. Con esto **las anónimas sobre una interfaz** (`new Runnable(){...}`, el caso más común) corren y verifican, y una **lambda contra una interfaz funcional propia** ya tipa y emite

Avanzado  **Sueltos de emisión cerrados** — `RuntimeVisibleAnnotations` (§4.7.16): se emite el atributo en clase/campo/método para las anotaciones **retenidas en runtime** (los `@interface` del fuente con `@Retention(RUNTIME)` + las conocidas del JDK; `@Override` / `@SuppressWarnings`, que son `SOURCE`, no se emiten), con los `element_value` taguados (String/primitivo/clase/enum/anidada/arreglo). **Plegado de constantes** (§15.28/§15.29): una `static final int` como etiqueta de `case` —o aritmética constante `case 1 + 4`, o una constante **cualificada** `case K.A` de otra clase que se **inlinea**— se pliega a un literal (mapa `campo → valor` a punto fijo, para constantes que refieren a otras). **Formas comprimidas del `StackMapTable`** (§4.7.4): cada frame se serializa en la forma más compacta según el anterior (`same / same_locals_1 / append / chop`, con `full_frame` de respaldo) — puro ahorro de bytes, validado por el verificador

 **Éxito: logrado** — el `javac` propio compila objetos, bucles y excepciones, y el `.class` **corre en tu JVM** (`fib(10)=55`, `fact(5)=120`, `new M(10); m.add(5); m.get() → 15`) y **pasa el verificador JVMs-estricto**, que hace *cross-check* de nuestra `StackMapTable` contra su propia inferencia.

B4 **Compilador robusto** **núcleo cerrado** **Avanzado**

Avanzado *  **Overload resolution** (**hecho en B2**) y chequeo de override — **pasada** *Check propia* (el `Check.java` de javac: bien-formación de las declaraciones, no de los usos). Cubre las cuatro reglas de §8.4.8 sobre firmas override-equivalentes* (mismo nombre y mismos parámetros **borrados**, §8.4.2): no sobrescribir un `final`, no cruzar la frontera `static` / instancia, no **reducir** la visibilidad, y **retorno covariante** — idéntico para primitivos, subtipo para referencias—. Más §8.1.1.1: una clase concreta tiene que implementar todo lo `abstract` que hereda, con las jerarquías que tocan un tipo **externo exentas** (de esos solo conocemos los miembros que aparecieron en alguna firma, así que una ausencia no prueba nada).  **Y ahora** `@Override` sin override (§9.6.4.4) —se reporta un `@Override` que no sobrescribe ni implementa nada; el chequeo es **sound**: solo dispara cuando el único ancestro externo es `Object` (cuyo set de métodos conocemos) y el método no es uno de los suyos (`toString` / `equals` / ...), y con un supertipo del JDK se calla— y el **throws más ancho** (§8.4.8.3) —una excepción **chequeada** que lanza un override tiene que ser subtipo de alguna del método sobrescrito—

Avanzado  **Inferencia** desde los argumentos *y desde el target type* — la atribución pasó a tener modo checking (`attrib_expr_to`), no solo síntesis: el tipo que el contexto espera baja a la expresión y es lo único que puede instanciar una variable que **no aparece** en los argumentos (`Caja<String> c = fabricar();`). Vale también para el **diamante** (§15.9.3), que antes pasaba por indulgente — `new Caja<>()` daba un parametrizado con cero argumentos, que se compara con cualquier cosa sin comparar nada— y ahora infiere de verdad, del target y del constructor. La **incorporación** (§18.3) arbitra entre las dos fuentes: un target que contradice a los argumentos se descarta, para que el error lo reporte el chequeo de asignación y no una inferencia inventada. Contextos cubiertos: los tres de asignación (§5.2 — inicializador, `return`, `=`) y ahora el de **argumento de llamada** por el algoritmo de **dos fases** (§15.12.2.6): la fase 1 resuelve la sobrecarga con los tipos provisionales/lenientes de los argumentos, y la fase 2 —resuelta ya— re-atribuye cada argumento **poly** (lambda, method ref, diamante) con el tipo de su parámetro como target. Es lo único que tipa el cuerpo de una **lambda-argumento** (`call(x -> x + 1)`) —sin él quedaba **Unresolved** y el emisor no podía bajarla— e instancia un **diamante-argumento** (`take(new Box<>())` infiere `String`); el diamante sin target se deja **crudo** para que la aplicabilidad lo acepte unchecked.  **Y la desambiguación de sobrecargas por el argumento poly** (§15.12.2.5): un arg lambda se tipa **Unresolved** (indulgente), así que sin esto sería aplicable a **toda** sobrecarga con un parámetro funcional; se descartan las incompatibles con la **forma** de la lambda —aridad del SAM y valor-vs-void—, así `f(x -> x + 1)` elige `Function` sobre `Consumer` (antes daba ambiguo). Cola menor: la misma disambiguación para method refs* (por aridad)

Avanzado  **Análisis de flujo (Flow)** — **asignación definitiva** de locales (Cap. 16, con los flujos *when-true/when-false* de `&&` / `||` / `!`), las reglas de variables `final` (conjunto *definitely unassigned*: no reasignar, asignación única) y **alcanzabilidad** (§14.21: sentencias inalcanzables), con `break` / `continue` **etiquetados** (§14.15/§14.16: la pila de contextos lleva la etiqueta a la que salta cada `break`)

Avanzado  **Desugar** — sentencias (`for-each` → `for` / `while` + `Iterator`, `try`-recursos → `try/finally` + `close()` con la **excepción del `close()`**) **suprimida** vía `addSuppressed` (§14.20.3.1), `assert` → `if(!$assertionsDisabled && !cond) throw` con su campo sintético y el `<clinit>` que lo calcula de `C.class.desiredAssertionStatus()` (§14.10), `++` / `--` **en descarte** → `x += 1` (§15.14.2), **switch-expresión** en cola → switch-sentencia que asigna (§15.28, con `yield` desde bloques/bucles vía `break` **etiquetado** sobre el switch generado), **switch sobre `String`** → dos switches sobre `int` con `hashCode` + `equals` (§14.11), **switch sobre `enum`** → switch sobre un `$SwitchMap` sintético (`int[]` indexado por `ordinal()`), alojado en una clase anidada `C$1` y poblado en un `<clinit>` con `try/catch NoSuchElementException`, **fiel a javac** §14.11), * **switch** con patterns (*incl. deconstrucción de records, que extrae cada componente por su accessor y es recursiva*) → cadena de `instanceof` en un bloque **etiquetado** (§14.11.1: cada brazo es un **if suelto** —no `else if`— para que una **guarda que falla** caiga al **case** siguiente; el binding se materializa con un `cast`, y un selector `null` sin `case null` lanza **NPE**) y expresiones (concat de `String` → `StringBuilder`, `x op= y` → `x = (T)(x op y)`), **varargs** → `array` en el call site). Habilitado por una **síntesis a nivel clase nueva**: `Member::StaticInit` (los bloques `static { }` ya no se descartan), tabla de símbolos **mutable** en el desugar, el `enum extends Enum` implícito y el **binding de variables de patrón** en `Attribute/Flow`. Además **materializa los miembros implícitos de un `record`** (§8.10.3: campos, constructor canónico y accessors, respetando lo declarado a mano) —sus símbolos ya venían de `enter`, pero sin cuerpo en el AST el `.class` los referenciaba **sin declararlos**, y la deconstrucción llamaba a un método inexistente. La síntesis de estos miembros (`enum/record/assert`) reencuentra el símbolo del tipo por su **FQN con paquete** (§7.1) —el desugar lo armaba sin el paquete, así que un `enum/record` en un **paquete nombrado** perdía **toda** su maquinaria (sin constantes/ `values()` / `valueOf()` / `$VALUES`): bug #21, ya cerrado con test—. El inlining del `finally` resultó ser del **codegen**, no de acá. Queda como **asimetría** que varias de estas reescrituras producen construcciones que el emisor **todavía no soporta** (`instanceof`, `new int[]`): desde que hay barrera, eso **falla** en vez de salir roto. Aparte (no son desugar, van en su propia pasada): el **boxing** es `TransTypes*` ( ítem aparte) y las lambdas/ `invokedynamic` son `LambdaToMethod`

Avanzado  **TransTypes** — **boxing/unboxing dirigido por tipo** (§5.1.7/§5.1.8) — el `TransTypes` de javac, acotado al **boxing** (la *erasure* de genéricos ya la resuelve el sistema de tipos por *erased ids*). Dos capas: (1) **Attribute lo acepta** — el contexto de asignación (§5.2) permite `int` → `Integer` / `Number` / `Object` (box + *widening* de referencia) y `Integer` → `int` / `long` / `double` (unbox + *widening* primitivo), sumado a la fase de boxing del *overload*; (2) una **pasada propia** (tras Flow, antes del desugar) que recorre el árbol tipado e **inserta la conversión** en cada sitio: primitivo donde se espera referencia → `Integer.valueOf(v)`, `wrapper` donde se espera primitivo → `v.intValue()`. Contextos: inicializador de `local` / campo, `return`, `=`, **argumentos** (con el tipo de parámetro resuelto — un genérico borrado *boxea*), **operandos** de aritmética/relacionales/lógicos (`i + 1` → `i.intValue() + 1`), índice de array, ramas del ternario, y el **cuerpo de una lambda** contra el retorno de su SAM. Las inserciones quedan sin decorar y las **materializa la re-atribución**; el emisor saca de ahí `invokestatic valueOf` / `invokevirtual xxxValue`. Cierra el caso insignia de las *poly expressions*: `Function<Integer,Integer> f = x -> x + 1` de punta a punta (el `int` del cuerpo se *boxea* al `Integer` del SAM, validado por el **verificador estricto** — ejecutar `valueOf` / `intValue` es de la JVM (KajijDK), track aparte—. *  **Y el wrapper→primitivo más ancho** (`Long l = anInteger`): tras `intValue()` (`int`) se inserta un `Cast` al target, que el emisor baja a `i2L` / `i2d`. Cola menor: el `==` / `!=` con *unboxing dirigido* (se dejó por la identidad de referencia entre dos wrappers*)

Avanzado  **Miembros implícitos del `enum`** (§8.9.3) — constantes como campos, `$VALUES`, el constructor privado (`(String, int)` y `values()` / `valueOf()`). Destruirlo pidió cerrar antes la **invocación explícita de constructor** (`super(...)` / `this(...)`, §8.8.7.1), que **no existía**: el parser la producía, la atribución la rechazaba, y sin ella no se puede heredar de ninguna clase con constructor parametrizado. Dos desvíos deliberados de javac, ninguno semántico: `values()` copia con un **bucle** en vez de `$VALUES.clone()` (lo que importa de `clone()` es devolver un array fresco), y `valueOf` compara contra los nombres literales en vez de delegar en `Enum.valueOf(Class, String)`, que va por reflexión. Se sumó `java.lang.Enum` a `bootstrap/ + boot/`: sin ella el `.class` emitido no se podía ni verificar ni correr.  **Y ahora el `enum` con constantes parametrizadas** (`ROJO("rojo")`, §8.9.2): a cada **constructor propio** se le antepone (`String $name, int $ordinal`) tanto en el AST como en el **Resolved del símbolo** (la re-atribución no re-resuelve firmas, y sin eso `new E("A", 0, ...)` no encontraría el ctor), y se le inyecta `super($name, $ordinal)` de cabecera —o se le **propaga** a un `this(...)` delegado—; cada constante se construye con sus argumentos (`new Color("ROJO", 0, "rojo")`), y el ctor (`(String, int)`) por defecto solo se sintetiza si el usuario no declaró uno. Corre de punta a punta (un `enum Size { SMALL(1), LARGE(3); ... }` da 4) y verifica

Avanzado ✓ **Inicializadores de campo** (§8.6/§12.4.2) — `int v = 7;` se vuelve una sentencia, unificada en **orden de fuente** con los bloques `{ }/static { }:` las de instancia al frente de cada constructor, las estáticas al `<clinit>`. Hasta ahora **no llegaban al** `.class`: un campo con inicializador compilaba a un campo en cero, y el emisor ni siquiera generaba `<clinit>` —así que el `$VALUES` de un `enum`, el `$SwitchMap` de su `switch` y el guard del `assert` quedaban declarados y vacíos—

Avanzado ✓ **Generación de StackMapTable** — hecha en B3 (ver ahí): la pila de operandos se lleva **tipada**, no como altura, y el tipo `uninitialized` (tag 8) identifica cada objeto por el offset de su `new`

✓ **Éxito: logrado** — compila programas con sobrecarga, herencia y flujo no trivial. Los siete frentes están cubiertos; quedan **colas menores** documentadas en cada ítem (`@Override` sin override y `throws` en el chequeo, el `target type` en posición de argumento, el `enum` con constantes parametrizadas) — ninguna bloquea el criterio.

B5 Cumbre del compilador ✓ núcleo cerrado Cumbre

Cumbre ✓ **Genéricos** — `type erasure`, `wildcards` (containment), subtipado e inferencia por argumentos ya funcionan

Cumbre * ✓ **Inferencia por target type*** — hecha en B4 (modo checking en la atribución; el diamante incluido). Las **poly expressions** ya se tipan y emiten (lambdas y method refs contra su interfaz funcional, con `invokedynamic`). El **boxing dirigido por tipo** (TransTypes), el `target type` en posición de **argumento de llamada** (dos fases §15.12.2.6) y la **capture conversion*** (§5.1.10) ya están **cerrados** (ítems aparte)

Cumbre ✓ **sealed / non-sealed + exhaustividad de switch** (§8.1.1.2/§9.1.1.4/§8.1.6/§14.11.1.1) — el parser reconoce las keywords **restringidas** `sealed / non-sealed` (con `lookahead` que las distingue de un tipo/campo llamado `sealed`) y la cláusula `permits`; el grafo de símbolos gana la lista de **subtipos autorizados** (`permitted`), con el `permits` **implícito** (§8.1.6) rellenado de los subtipos directos de la unidad, así el resto del pipeline ve el conjunto completo. **Check** valida las reglas de buena formación: un subtipo directo de un `sealed` debe ser `final / sealed / non-sealed` (o implícitamente `final`: `enum / record`), un `permits` explícito solo puede listar subtipos **directos** y todo subtipo debe estar autorizado, y `non-sealed` exige un super `sealed`. Y la **exhaustividad**: una `switch`-**expresión** (siempre) y una `switch`-***sentencia con patterns** `case null sin default` **deben cubrir todos los valores — la jerarquía sealed** recursiva **hasta las hojas, todas las constantes de un enum, y true / false de un boolean (JEP 455) —, y un patrón con guarda (when) no cubre. El .class emitido lleva el atributo PermittedSubclasses (§4.7.31), re-parseado por la JVM propia: el tipo queda realmente sellado. Destapó de paso un bug del parser — case RED -> 1 se leía como la lambda RED -> 1; una etiqueta constante es una conditional-expression, no una assignment-expression*—, ya cerrado**

Cumbre * ✓ **Capture conversion*** (§5.1.10) — la wildcard capture con **variables de captura frescas**, no una aproximación a cotas. Un `RType::Capture{id, upper, lower}` lleva sus cotas **inline** (sin tocar la tabla, que `Attribute` tiene inmutable) y un `id` fresco (contador con mutabilidad interior) que la hace **distinta** de otra captura igual —la freshness del §5.1.10—. La conversión (`capture`) transforma `G<?...>` en `G<...CAP_i...>`: `? / ? super B` → cota superior `B` la del parámetro (`Object`); `? extends B` → `glb(B, U_i)`; y `? super B` guarda además la cota **inferior** `B`. Se aplica en **acceso a miembros** (el receptor se captura **una vez**, así todas las sustituciones de la llamada comparten las mismas variables) y en **argumentos de invocación**. El subtipado ganó las dos reglas: `CAP <: t` por su cota **superior**, y `s <: CAP` solo por la **inferior** (`? super`). Efectos observables: `List<?>.get()` → `Object`, `List<? extends Number>.get()` → `Number`, el **lado escritura** de `? super Integer` acepta un `Integer` pero **no** un `Object` (por campo, donde el chequeo es estricto), dos capturas del mismo `?` **no unifican** (`a.val = b.val` se rechaza), y el caso insignia `swap(List<?>) → swapHelper(List<T>)` ahora **infiere** `T` = la captura (para lo que se hizo aplicable un `<T> m(Box<T>)` a un argumento cuyo tipo lo fija la inferencia, no la invariancia). La erasure de una captura es la de su cota superior, así que el emisor y el verificador la ven como su upper bound*. ✓ **Y el lado escritura por llamada** (cerrado en B6): la aplicabilidad **sustituye el receptor** en los params, así `list.add(x)` sobre `List<? super X>` también se chequea estricto —antes solo por campo—

Cumbre * ✓ **Bridge methods (§8.4.8.3 / §15.12.4.5)** — **los métodos puente sintéticos que hacen que la sobrescritura funcione a nivel de bytecode** cuando la erasure difiere. El emisor, tras las clases reales, detecta por cada método propio concreto de instancia los métodos de un supertipo que **sobrescribe** —los params de aquél **sustituídos** (`T := Integer`) y borrados coinciden con los suyos (§8.4.2)— cuya erasure* **sin substituir** difiere, y sintetiza uno con el **descriptor borrado del supertipo** (`ACC_BRIDGE + ACC_SYNTHETIC`) que reenvía al real: `this` + los argumentos con un `checkcast` al tipo angosto + `invokevirtual` al método real + el `return` (el subtipo del retorno es asignable). Cubre las dos causas: el **parámetro genérico borrado** (`class INode extends Node<Integer>` con `set(Integer) → puente set(Object)`), y el clásico `Foo implements Comparable<Foo> → compareTo(Object)`) y el **retorno covariante** (`A.get():Object / B.get():String → puente Object get()`). Se dedup por descriptor y no pisa un método real ya presente. Verificado de punta a punta: el `.class` lleva el puente con sus flags, **verifica** (`checkcast + invokevirtual`), y el **despacho** funciona —`a.pick()` visto como `A.pick():Object` sobre un `B` corre el puente que reenvía a `B.pick():String`—. Acotado a **clases** (en una interfaz el puente sería un `default`, aparte)

Cumbre ✓ **Módulos** (§7.7 / §4.7.25) — el sistema de módulos (JPMS) del lado del **compilador**: parseo de un `module-info.java` y emisión de su `module-info.class`. El parser reconoce las keywords **restringidas** (`module`, `open`, `requires`, `exports`, `opens`, `uses`, `provides`, `to`, `with`, `transitive` — todas identificadores salvo `static`) y arma la `[open] module nombre { ... }` con las **cinco directivas**: `requires [transitive] [static]`, `exports [to]`, `opens [to]`, `uses, provides ... with` (con la sutileza de `requires transitive`; donde `transitive` seguido de `;` es el **nombre** del módulo, no el modificador). El **emisor** produce un `module-info.class` fiel: `ACC_MODULE`, `this class = module-info`, sin super/campos/métodos, y el atributo `Module` (§4.7.25) con los flags (`ACC_OPEN / ACC_TRANSITIVE / ACC_STATIC_PHASE`) y el `requires java.base` **mandated** implícito; el pool ganó `CONSTANT_Module / CONSTANT_Package` (nombres de módulo con **puntos**, de paquete/servicio en forma **interna** con `/`). **Check** valida la buena formación (§7.7): directivas **duplicadas** (mismo módulo/paquete/servicio), implementación repetida en un `provides`, y auto-`requires`. Verificado re-parseando el `module-info.class` con la JVM propia (que ya leía los tags 19/20). La **resolución del grafo** de módulos —legibilidad, accesibilidad entre módulos— es de tiempo de ejecución (JPMS), track de la JVM aparte

✓ **Éxito: logrado** — los grandes frentes de **genéricos** y de **Java SE 25** están cerrados: erasure/wildcards/inferencia, captura, sealed+exhaustividad, bridge methods y módulos. La **equivalencia plena** con `javac` (emitir `Signature`, inferencia completa del Cap. 18) es cuestión de **fidelidad**, y se trackea en B6.

B6 Fidelidad de `javac` pendiente CumbreCumbre ✔ **Atributos de class-file** — se emiten

Code / SourceFile / StackMapTable / BootstrapMethods / RuntimeVisibleAnnotations / RuntimeVisibleParameterAnnotations / AnnotationDefault / PermittedSubclass y ✔ **Signature**. ✔ **Signature** (§4.7.9) —el de mayor valor, ya **cerrado**: la firma **genérica** que la erasure borra del descriptor, emitida en **clase** (`<T:Ljava/lang/Object;>Ljava/lang/Object;`), **método** (`<T:…>(TT;TT;)`) y **campo** (`Ljava/util/List<Ljava/lang/String;>;`, `TT;`), con las cotas (interfaz tras un `:` extra), los wildcards (`+/-/`) y solo cuando el elemento **usa genéricos**; sin él una clase/método/campo genérico perdía sus genéricos en el `.class` y la reflexión + la compilación separada los veían borrados. ✔ **InnerClasses** (§4.7.6) —también **cerrado**: cada `.class` lista su **cadena de enclosing** (él mismo si es anidado + sus ancestros) y las clases que **contiene**, clasificando **miembro** (con dueño y nombre) / **local** (`Outer$1L`: sin dueño, con nombre) / **anónima** (`Outer$1`: sin dueño ni nombre) por el sufijo del binary, con los flags de la declaración; es lo que la reflexión necesita para `getEnclosingClass` / `getDeclaringClass` / `isAnonymousClass`. ✔ **EnclosingMethod** (§4.7.7) —el **complemento** de InnerClasses para local/anónimas, también **cerrado**: el desugar registra, al **levantar** la clase, el (nombre, descriptor) del método/constructor envolvente (o `None` en un inicializador → `method_index` 0), y el emisor pone `class_index` = la clase envolvente y `method_index` = ese `NameAndType`; con esto `getEnclosingMethod` / `getEnclosingConstructor` andan—. ✔ **Record** (§4.7.30) —también **cerrado**, fiel: un `record` ahora extiende `java.Lang.Record` (agregada a `boot/`, extraída del JDK), lleva `ACC_FINAL` y el atributo `Record` con sus componentes (nombre + descriptor, + `Signature` por componente genérico), lo que la reflexión exige para `isRecord()` / `getRecordComponents()`; de paso destapó y cerró dos bugs latentes —el parser no tomaba un **record genérico** (`record Box<T>(...)`) y el descriptor de una **variable de tipo** salía `LT;` en vez de la erasure `Ljava/lang/Object;` (no verificaba)—. ✔ **NestHost / NestMembers** (§4.7.28/§4.7.29) —también **cerrado**: el nest (Java 11) que da acceso privado entre anidadas sin puentes sintéticos; cada clase **anidada** lleva `NestHost` a su **top-level**, y la top-level un `NestMembers` con **todas** sus anidadas (miembro/local/anónima, transitivo) —un nest **plano** por árbol de anidamiento—. ✔ **MethodParameters** (§4.7.24) —también **cerrado**: los **nombr**es de los parámetros formales (+ `ACC_FINAL` si se declaró `final`, y `ACC_SYNTHETIC` para las capturas `this$0` / `val$x`), lo que hace que la reflexión (`Method.getParameters()`) devuelva los nombres reales en vez de `arg0` / `arg1`; incluso un `record` conserva los nombres de sus componentes en el ctor canónico—. ✔ **Exceptions** (§4.7.5) —también **cerrado**: las clases de la cláusula `throws` de cada método (los índices `Class` de las excepciones **chequeadas** que declara lanzar), lo que hace que `Method.getExceptionTypes()` funcione y que la compilación separada vea las `checked exceptions`; va tanto en un método concreto (junto al `Code`) como en uno **abstracto/nativo** (sin `Code`); verificado diferencialmente contra el `javap` de Temurin (lee el atributo con las clases correctas y en orden). ✔ **LineNumberTable** (§4.7.12) —también **cerrado**: anidado en el `Code`, mapea offset de bytecode → línea del fuente (el emisor marca la línea al entrar a cada sentencia, deduplicando por pc y por línea; el `super()` implícito de un ctor se mapea a la línea de su declaración), lo que da los **números de línea en los stack traces** y habilita breakpoints por línea en un debugger; verificado contra `javap -L` de Temurin. ✔ **LocalVariableTable** (§4.7.13) —también **cerrado**: anidado en el `Code`, da los **nombr**es (+ descriptor + slot + rango de vida) de `this`, los parámetros y los locales del cuerpo, lo que hace que un debugger muestre los nombres reales. El emisor mantiene una **pila de scopes** que espeja la de la pasada 2 (que reusa slots al salir de un bloque): así dos locales que **comparten slot** en bloques disjuntos dan rangos **no solapados** (verificado contra `javap -L`, con `a/b` en el mismo slot y rangos disjuntos, igual que `javac`); los locales **muer**tos (rango de largo 0) se descartan, también como `javac`. Cubre método/`Block` / `for` / `switch` / `synchronized` / `try-catch-finally`; quedan afuera los pattern bindings (`instanceof` / `switch`) —incompletos pero **nunca solapados**—. ✔ **ConstantValue** (§4.7.2) —también **cerrado**: un `static final` con inicializador de **expresión constante** (§15.29) lleva el atributo (`Integer` / `Long` / `Float` / `Double` / `String`, coercionado al tipo del campo), que la JVM asigna al **preparar** la clase; el desugar lo deja **sin bajar** al `<<Linit>` (no se inicializa dos veces, igual que `javac`) y así las constantes se pueden inlinear en compilación separada (§13.4.9); un evaluador acotado pliega literales con `+/ -` unario (las expresiones aritméticas complejas caen al `<<Linit>`, correcto pero sin inlinear); verificado contra `javap` de Temurin. ✔ **RuntimeVisibleTypeAnnotations** (§4.7.20, JSR 308) —el último atributo, **cerrado**: ✔ **Fase 1** las anotaciones sobre **parámetros de tipo** (`class C<@Foo T>` target `0x00`, `<@Foo T> void m()` `0x01`); ✔ **Fase 2** las anotaciones **líder** de declaración ruteadas por su `@Target` (`@NonNull String` sobre **campo** `0x13`, **retorno** `0x14`, **parámetro** `0x16`, con exclusión del `RuntimeVisibleAnnotations` para las `TYPE_USE`-only); ✔ **Fase 3** los usos **anidados** con su `type_path` reconstruido por el parser —argumentos de tipo (`List<@A String>` → `[TYPE_ARGUMENT(0)]`), wildcards* (`? extends @A T` → `[..., WILDCARD]`) y arrays con la **dirección** correcta del path (`int @A []` → path vacío en la dimensión externa, `List<@A X>[]` → `[ARRAY, TYPE_ARGUMENT(0)]`), `extends` / `implements` (`0x10`, `supertype_index` `0xFFFF` / índice de interfaz), **cotas** de parámetro de tipo (`0x11` clase / `0x12` método, con el `bound_index` que arranca en 1 cuando la primera cota es una interfaz —el `Object` implícito ocupa el 0—, como `javac`), `throws` (`0x17`) y los **targets de Code** —cast (`0x47`, offset del `checkcast` + `type_argument_index`), `instanceof` (`0x43`), `new` (`0x44`, objeto y array, con el offset del opcode / del inicio de la creación) y **variable local** (`0x40`, con la **tabla de rangos de vida** {start_pc, length, index}, el mismo rango que el `LocalVariableTable`, y filtradas por `@Target(TYPE_USE)`)—, estos cuatro en un `RuntimeVisibleTypeAnnotations` **anidado en el Code**. Para retener la anotación por posición de tipo el parser acumula un buffer `pending_type_annos` (el `Type` del AST no tiene slot) que las declaraciones/expresiones drenan con su `type_path`, y el emisor la rutea al target + offset correcto. **Todo verificado byte-fiel** contra el `javap` de Temurin (parámetros de tipo, cotas, extends/implements, throws, field/return/param, y los cuatro targets de `Code`). Con esto **RVTa queda cerrado** y no quedan atributos de fidelidad **reflexiva** pendientes —solo variantes menores de bajo valor (`RuntimeInvisible{,Type,Parameter}Annotations`, `LocalVariableTable`)—

Cumbre ✔ **Inferencia completa (Cap. 18)** — cerrada, con **barrido diferencial** contra `javac` **real** como evidencia. ✔ **Lado escritura de la captura por llamada** — **cerrado**: la aplicabilidad ahora **sustituye el receptor** en los parámetros (§15.12.2.2), así una variable de tipo de **clase** pasa a su argumento (incluida la variable de **captura** de un `? super B`), y `list.add(x)` sobre `List<? super X>` se chequea **estricto** (`add(X)` vale, `add(Object)` no); las variables de tipo de **método** las deja la sustitución como están, para que la inferencia las resuelva. ✔ **lub con intersección** (§4.10.4) — **cerrado**: un `RType::Intersection(A & B)` cableado por el sistema (subtipado `A&B <: t` sii algún miembro; `s <: A&B` sii todos; `erasure` = primer miembro, la **clase**), y el `lub` se computa por **supertipos borrados comunes mínimos** (así `Comparable` no se pierde por invariancia) recuperando la forma parametrizada cuando hay uno solo; ahora `lub(Integer, Long) = Number & Comparable`, y el ternario/inferencia se puede usar como cualquiera de los dos. ✔ ✔ **Y el solver transitivo (§18.3/§18.4)** — **cerrado**: **se siembran las cotas declaradas de cada variable** (`<C extends List<T>> => C <: List<T>`), la incorporación **cierra los bounds a punto fijo cruzando** `S <: a` con `a <: U` (y reduciendo lo derivado, que puede fijar otra variable), y la resolución **instancia en orden de dependencia sustituyendo lo ya resuelto (un ciclo se fuerza junto)**. Así una variable sin cota directa se deduce **fluyendo por la de otra**: `<T, C extends List<T>> T first(C c)` sobre un `List<String>` infiere `T = String` (antes caía a `Object`). De paso destapó un bug de `enter`: las cotas de un **parámetro de tipo de método se resolvían en global** en vez del scope propio del método, así que un `<C extends List<T>>` no veía ni al hermano `T` ni a las clases del paquete. ✔ Y el containment de `? super X` (§18.2.3) — el caso `Super` de `reduce_arg`, que se caía: `List<? super X>` como **parámetro** acota `X` por arriba (`sup(aListOfAnimal)` con `<T> T sup(List<? super T>)` infiere `T = Animal`). ✔ Y la detección de `false` (§18.5.1) en su **caso claro** — dos igualdades incompatibles sobre una misma variable (`m(List<Foo>, List<Bar>)` sobre `<T> void m(List<T>, List<T>)`) hacen fallar la inferencia y se reporta el error (en métodos propios; sobre un externo se mantiene la indulgencia, que no modela toda firma del JDK). ✔ Y el argumento fuera de la cota declarada (§4.5.1/§18.3): `S <: a` con `S` fuera de la cota de `<a extends B>` (`m("x")` sobre `<T extends Number>`) ahora hace

fallar la inferencia —la aplicabilidad, indulgente con la variable de método, lo dejaba pasar—. Con esto el `false` de §18 queda cubierto en lo que importa. Las contradicciones cota-inferior/superior derivadas del target (`Integer n = id("hola")`), donde el contexto impone `T <: Integer` y el argumento `String <: T` se dejan a propósito al chequeo de asignación, que da mejor mensaje; reportarlas también en la inferencia las duplicaría. ✓ Y las variables de inferencia frescas (§18.1) con solving combinado de anidadas (§18.5.2.1): cada invocación genérica introduce variables frescas (`RType::InferVar`, no el propio parámetro de tipo), y una llamada genérica anidada como argumento (`pick(empty(), strs())`) ya no se resuelve aislada —lo que daba `empty() → Box<Object>` y rechazaba la llamada—: sus variables entran en el mismo conjunto de constraints que las de la externa, así `empty()` toma su `E` del contexto (`E = String`) y el nodo se re-tipa. ✓ Y la aplicabilidad por inferencia (§18.5.1) con desempate de sobrecargas (§18.5.4): un overload genérico cuya inferencia de argumentos es insatisfacible (`m(T, List<T>)` sobre `(List<String>, List<String>)`): el 2º arg fija `T = String`, el 1º pide `List<String> <: T` ya no es aplicable —lo detecta un `hard_false` sobre los bounds sin podar, gateado solo para desambiguar entre varios candidatos—, así que el desempate elige el mismo método que javac. Con esto, y completando la tabla de containment del §18.2.3, un barrido de 13 casos borde (desempate, capturas, wildcards, functional-interfaces, lub/glb) matchea a javac uno a uno. ✓ Y las tres esquinas que faltaban —que se creían no-op por capturar en el sitio de uso, y resultaron ser tres divergencias observables reales, las tres de indulgencia (aceptábamos programas que javac rechaza, o rechazábamos uno que acepta)—, cerradas y verificadas contra Temurin 21: (a) §18.3 capture-in-bounds —un método que devuelve `G<? extends T>` toma su `T` del target (`Box<? extends Number> b = make() ⇒ T = Number`); antes no se miraba dentro del wildcard del retorno, `T` caía a `Object` y se rechazaba una asignación válida (nuevo `reduce_target_arg: ? extends inner` contra `? extends X ⇒ inner <: X, y ? super` análogo). (b) §18.4 —la variable de una llamada anidada que vive solo bajo un wildcard del retorno (`take(make())`) se resuelve por sus cotas (`⇒ su` declarada `Object`), no heredando el target externo; la clausura transitiva ya no le pasa cota superior a una variable de inferencia pelada, así `take(make())` con target `Number` falla como javac (antes lo aceptábamos). (c) §5.1.10 —pasar un argumento `G<? extends X>` a un parámetro invariante `G<α>` captura (`α = CAP` fresca); si el target además fija `α = T` concreto son igualdades incompatibles y la llamada se rechaza (`reduce_arg` gana las arms de captura contra param invariante, con cota concreta para que el conflicto sea proper). Un barrido diferencial de 9 casos anidados (capture/wildcard/nido) contra Temurin 21 matchea uno a uno. El único resto del formalismo —el second attempt literal del §18.4, la variable de tipo `Y` de rescate— es un no-op observable verificado: la capture conversion* (§5.1.10) en el sitio de uso ya produce la `Y` equivalente, así que no hay ningún programa que rescate (implementarlo pediría mantener bounds `G<α> = capture` en el conjunto, reescritura arquitectónica para cero cambio observable). La inferencia del Cap. 18 queda cerrada, con barrido diferencial contra javac real como evidencia —ya no por argumento de equivalencia—

Cumbre ✓ **Diagnósticos y recuperación de errores (cerrado)** — ✓ **recuperación del parser**: en vez de abortar en el primer error de sintaxis, el parser entra en **panic-mode** y **resincroniza** —a nivel **unidad** (salta al comienzo de la próxima declaración de tipo, rastreando llaves), **miembro** y **sentencia** (salta al próximo `;/` del cuerpo)—, acumula **todos** los errores en un `Vec<Error>` y **sigue** construyendo el AST con las partes buenas; así una unidad con tres declaraciones rotas reporta los **tres** errores en una corrida y aún parsea los miembros/tipos válidos de alrededor. La **semántica** ya reportaba varios (Enter/Attribute/Check/Flow acumulan), y `--check` ahora lista los sintácticos **primero** y luego los semánticos. ✓ **Frame de render tipo javac (cerrado)**: el CLI ya no imprime un escudo `línea:col: error: msg` sino el marco de `javac` —`archivo:línea: error: mensaje` (la columna se transmite con el **caret**, no en el texto), la **línea del fuente**, una línea con el `^` bajo la columna (preservando los tabs del prefijo para alinear), las **notas** indentadas, y un resumen `N error[s]` al final—; el tipo `Error` ganó un campo `notes: Vec<String>` que el render imprime. ✓ **Mensajes ricos (cerrados los tres más comunes)**: el **cannot find symbol** lleva las sub-líneas `símbolo: / ubicación: (método fro(int) / clase String`, con las etiquetas alineadas en los cuatro sitios —método, campo, nombre pelado, *method ref*—; los **candidatos de sobrecarga descartados** (§15.12.2) se listan cuando el nombre existe pero ninguna sobrecarga aplica, cada uno con su firma (receptor sustituido) y el **motivo** (aridad distinta, o el primer argumento que no convierte, espejando `applicable`); y el **did-you-mean** sugiere el nombre más cercano en scope por **distancia de edición** (Levenshtein, umbral `long/3`) —método/campo del receptor, o locales + campos de la envolvente para un nombre pelado—. ✓ **Nodos de error en el AST (cerrado)**: la recuperación a nivel **expresión**, no solo estructural. Una expresión que no parsea se vuelve un nodo `ExprKind::Error` y la **estructura de alrededor sobrevive** —cableado en el inicializador de un `local` (la variable **igual queda declarada**) y en los **argumentos** de una llamada (`f(bad, ok)` no pierde `ok`)—; el parser registra el error, sincroniza a un límite de expresión (`, / ; / ) / ] / }` de profundidad 0) y sigue, y la atribución tipa el nodo `Unresolved` **sin** un error nuevo. Así `int x = @@@; int z = x + 1;` reporta **solo** el error de sintaxis: `x` queda declarado y `x + 1` **no cascadea** «no se encuentra `x`». Con esto el ítem queda **cerrado**; el único resto es puramente cosmético (los **mensajes ricos** de `javac` que faltan —candidatos de constructor, notas de `unchecked`—, de bajo valor)

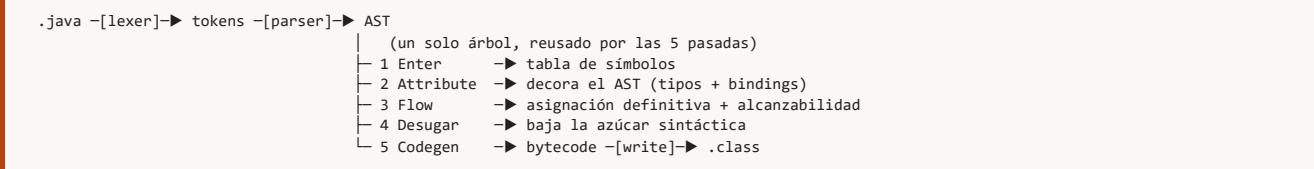
Cumbre **Esquinas de lenguaje** — **bridges en interfaz** (métodos `default` puente), evaluación **completa** de expresiones constantes (§15.29) fuera del `case`, precisión de exhaustividad con **record patterns anidados**, y refinamientos de asignación definitiva (un `switch` exhaustivo sin `default` que asigna una variable, hoy no se integra con Flow porque esa pasada no usa la tabla de símbolos)

Cumbre **Nivel herramienta** (opcional, fuera del núcleo del lenguaje) — `annotation processing` (APT, `javax.annotation.processing`), `-Xlint` y `javadoc`. Son parte del `javac` **como herramienta**, no del compilador de lenguaje; se listan para completitud

✓ **Éxito**: el `.class` emitido y los diagnósticos son **indistinguibles** de los de `javac` para el núcleo del lenguaje: se emiten **todos** los atributos, la inferencia cubre el Cap. 18, y los errores se **recuperan** en vez de cortar en el primero. Los cinco frentes grandes de Java SE 25 ya están (B5); esto cierra la brecha de **fidelidad** que queda.

Compilador — pasadas y estructuras internas

El compilador es **multi-pasada** sobre un **único AST**: se construye una vez (parser) y todas las pasadas lo recorren — la 1 lo lee para armar la tabla de símbolos, la 2 lo relee y lo **decora** con tipos y bindings. El objetivo es un compilador **completo** de Java SE 25; la semántica de cada regla está documentada y citada a la JLS 25 en `docs/semantica-jdk25.md`.



Estado: ✔ hecho 🚧 parcial 🔴 falta

Pasada	Qué hace	Hito	Estado
Lexer	texto <code>.java</code> → tokens	B0	✔
Parser	tokens → AST (cada nodo con su posición de fuente)	B1	✔
1 · Enter	declaraciones → tabla + grafo tipado (class finder real, resolución)	B2	✔
2 · Attribute	resuelve nombres + tipa + decora el AST (overload, genéricos, inferencia)	B2	✔
Check	bien-formación de declaraciones (override §8.4.8, abstractos §8.1.1.1) + usos (acceso §6.6, excepciones chequeadas §11.2)	B4	✔
3 · Flow	asignación definitiva (+ <code>final</code>) + alcanzabilidad	B4	✔
4 · Desugar	baja azúcar (sentencias, concat, compuesta, varargs, <code>++ / --</code> , <code>switch-expr</code> →sentencia, <code>switch</code> sobre <code>String / enum / patterns</code> , guard del <code>assert</code> , records, miembros del <code>enum</code> , inicializadores de campo), LambdaToMethod (lambdas/ <i>method refs</i> → <code>invokedynamic</code>), <code>record</code> por <code>ObjectMethods</code> , y la captura del entorno de las clases internas/locales/anónimas (<code>this\$0 / val\$</code> , con <code>hoist_anonymous</code>); el boxing va en <i>TransTypes</i> , su pasada aparte	B4	✔
5 · Codegen	AST decorado → bytecode → <code>.class</code> (v69): objetos, tipos anchos, flujo de control completo (<code>switch</code> , ternario, saltos etiquetados), excepciones, <code>synchronized</code> , <code>StackMapTable</code> e <code>invokedynamic</code> (lambdas, <i>method refs</i> , <code>record</code>)	B3	✔

Las pasadas no inventan código: **ordenan y anotan**. El orden lo imponen las dependencias — **Enter** antes que **Attribute** (por las referencias hacia adelante), Attribute antes que Flow y Desugar (necesitan tipos), Desugar antes que Codegen. La 1 lee “a lo ancho” (declaraciones), la 2 “a lo hondo” (cuerpos).

Estructuras intermedias

Además del AST, las pasadas comparten un **bolso de servicios globales** (la *Session*) y usan estado **transitorio** por recorrido. Dos que se olvidan siempre y duelen después: el **log de errores** (acumular, no frenar en el primero) y el **interner de nombres**.

Estructura	Rol	Vida
Tabla de símbolos	catálogo de todo lo declarado, con sus tipos	persistente
Arena de tipos (<i>interned</i>)	tipos resueltos, comparables por <code>TypeId</code> en O(1)	persistente
Interner de nombres	identificadores → <code>NameId</code> (comparación e identidad baratas)	persistente
Log de errores	acumula errores/warnings — no aborta al primero	persistente
Decoración del AST	cada nodo lleva su tipo y su binding in situ (salida de Attribute, estilo <code>JCTree.type / sym</code>)	persistente
Álgebra de tipos (<i>types</i>)	subtipado, erasure, sustitución, <code>lub / glb</code> — el <code>Types</code> de javac	persistente
Motor de inferencia (<i>infer</i>)	variables de inferencia + <i>bounds</i> + resolución (Cap. 18)	transitorio
Tabla de constantes	valores de expresiones constantes (JLS §15.29)	persistente
Class finder	carga los tipos externos (<code>.class</code> del classpath)	persistente
Env (contexto)	<i>scope</i> / clase / método / tipo esperado actuales	transitorio
Estado de Flow	<i>bitsets</i> de asignación definitiva y alcanzabilidad	transitorio
Estado de inferencia	variables de inferencia + restricciones (genéricos/lambdas)	transitorio

Representación: **arenas + IDs** (símbolos por `SymbolId`, tipos por `TypeId`, nodos por `NodeId`) en vez de punteros — la forma idiomática de Rust para grafos con ciclos, y coherente con el *metaspace* de la JVM, que ya referencia por `MethodId`/`Class ID`. La **tabla de símbolos** es un **árbol de scopes enlazados** que persiste (no una pila que se destruye al salir); el **Env** lleva solo el *scope* actual y sube/baja por esa cadena.

Fase C — Las bibliotecas (en Java, compiladas por tu javac)

C0 Núcleo de java.lang Base

Base `Object`, `String`, `System`, `wrappers`, `Math`, `StringBuilder`

✓ **Éxito:** un programa que usa `String` y `System.out` corre en tu JVM.

C1 Excepciones Base

Base Jerarquía `Throwable` / `Exception` / `RuntimeException`

✓ **Éxito:** lanzar y atrapar excepciones de la biblioteca propia.

C2 Colecciones e IO Avanzado

Avanzado `java.util`: `List`, `ArrayList`, `Map`, `HashMap`

Avanzado `java.io`: `InputStream` / `OutputStream` / `PrintStream`

✓ **Éxito:** un programa que usa `ArrayList` / `HashMap` corre.

C3 Cumbre de la biblioteca Cumbre

Cumbre Resto de `java.base` (`net`, `nio`, `time`, reflexión completa...)

✓ **Éxito:** cubrir lo necesario de `java.base` para programas reales.

Fase D — Herramientas (la “DK” = Development Kit)

No se construyen en bloque, sino que se van completando como subproducto de las otras fases.

Herramienta	Cuándo se logra	Horizonte
<code>javap</code> (desensamblador)	Se logra con el Hito A0	Base
<code>java</code> (lanzador de la JVM)	Se logra durante la Fase A	Base
<code>javac</code> (compilador)	Es toda la Fase B	Base
<code>jar</code> (empaquetador de <code>.class</code>)	Tras la Fase B	Avanzado
<code>jdb</code> , <code>javadoc</code> , <code>jlink</code> , <code>keytool</code>	Largo plazo	Cumbre

Fase E — Cerrar el círculo

El momento épico: las tres piezas funcionando juntas.

- **Cumbre** Compilar las bibliotecas (Fase C) con tu propio `javac` (Fase B)
- **Cumbre** Ejecutar un programa real que use esas bibliotecas en tu propia JVM (Fase A)
- **Cumbre** Conformance: comportamiento fiel a la especificación

✓ **Éxito:** un `.java` que escribís → lo compila tu `javac` → usa tus bibliotecas → corre en tu JVM, sin tocar nada del JDK de Temurin.

Más allá del JDK — la JVM como plataforma

La JVM no es un fin en sí mismo: es la **carrocería** de un proyecto más grande. Como construimos VM + compilador propios, controlamos el stack entero y podemos **inventar** más allá de lo que `javac` / HotSpot permiten. Estos dos tracks son extensiones aspiracionales apoyadas en las fases A–E.

Fase F — `burst`: el optimizador (rendimiento)

Módulo de optimización **opcional** atornillado sobre el intérprete ingenuo, que actúa de **chasis y oráculo de corrección**. `burst` debe dar resultados **idénticos** y se valida con **differential testing** (mismo programa por los dos caminos → `assert` de igualdad). Importante: un optimizador **no necesita JIT** — el intérprete tiene su propia caja de herramientas (típico 2–10× sin compilar a nativo).

Estado (2026-08-13): la caja de herramientas del intérprete se abrió primero y rindió lo prometido — **campos –86%, virtuales –45%, llamadas –40%** sin compilar nada, solo dejando de recomputar lo mismo—, y después **el JIT arrancó de verdad**: emisor x86-64 propio, memoria W^X , compilador del subconjunto frame-local y OSR, con **116× en bucles** (17,3 millones de opcodes interpretados → 620). El **differential testing** que este módulo exige no hubo que construirlo: el oráculo `green=os-gil=os` del proyecto **ya lo es**, con el JIT activo en green y apagado en os-parallel. Detalle por hito abajo; la disciplina de medición que lo sostiene está en `F-bench`.

`F-bench` Infraestructura de medición ☒ **hecha** Avanzado

Avanzado **Workloads** (`java/Bm*.java`): `BmLoop` (aritmética frame-local = el piso), `BmArray`, `BmInvoke` (`invokestatic`), `BmVirtual` (`invokevirtual` polimórfico), `BmField` (`new` + `get/putfield`). Valores esperados contrastados contra el `java` real del JDK 25, y un test **no-ignorado** los verifica para que los benchmarks no se pudran en silencio

Avanzado **Protocolo, aprendido a los golpes**: el ruido de `code layout` de esta máquina es **±3-12%, mayor que el efecto de casi cualquier cambio**. Demostrado con un experimento nulo: agregar un campo `HashMap` **sin usar** movió un control +3.4%. Por eso: workloads de **control de efecto-cero** (verificados *contando* la operación, no razonando), **cuadrado latino** con los binarios pre-compilados, medianas y mínimos, y la regla "si la dispersión intrarama es mayor que el efecto que declarás, todavía no lo mediste"

Avanzado **Medir antes de optimizar** — dos cuellos "obvios" resultaron falsos: el GC parecía el culpable de que `BmField` fuera 8.6× el piso (era **≤5%**), y el scan lineal $O(\#clases)$ de `class_name_at_mirror` parecía la gran palanca de `BmVirtual` (valía ~3%: hay **5 mirrors**, no cientos)

✓ **Éxito: hecho** — sin números honestos antes/después, todo el track es fe. `bench_baseline` (`#[ignore]`, `cargo test --release --lib bench_baseline -- --ignored --nocapture`) mide 5 workloads que aíslan una dimensión cada uno; los conteos de opcodes son exactos y reproducibles, así que **ns/opcode** compara entre workloads aunque los tiempos absolutos tengan ruido.

`F0` **Quickening** ☒ **hecho** Avanzado

Avanzado **El diagnóstico**: las tres optimizaciones tenían la **misma forma** — *un cache keyeado por `&str` alcanzado alocando un `String`, recomputado por ejecución en vez de por `call site`*. Y esas alocações no eran necesidad semántica sino **impuesto del borrow checker** (`class_of()` / `name()` ya devuelven `&str`; se clonaban solo para poder llamar métodos `&mut self` después). Por eso se pudo sacar el 86% sin tocar semántica: no se hizo el trabajo más rápido, **se dejó de hacerlo dos veces**

Avanzado **Campos** (`field_sites`): `field_offset` **reconstruía el layout entero de la clase** — un `Vec<(String,String)>` fresco, recorriendo la cadena de superclases — **en cada acceso**. Cache por (`MethodId`, `pc`): una celda `AtomicU64` por byte de código, sin hash ni strings. De paso cerró un peligro real: `layout_fields_ref` truncaba en silencio ante una superclase no cargada, devolviendo offsets **demasiado chicos**; sin cache eso solo causaba una escalación, **con** cache habría inmortalizado un offset incorrecto — ahora solo se cachea si la cadena se recorrió entera

Avanzado **Llamadas** (`call_sites`): callee resuelto, `vtable_slot` del tipo estático, anchos de slots (antes un `Vec` fresco por llamada), y la cadena de **~7 comparaciones de string** contra nombres de intrínsecos reemplazada por un enum `Intrinsic` **en el `MethodBody` del callee** — no en el sitio — porque por `vtable` el sitio no puede saber la respuesta: depende del receptor. El bit de "ya inicializada" solo se pone con la clase en `Done`, así que `Erroneous` sigue tirando `NoClassDefFoundError` en cada uso

Avanzado **Pool de frames**: reusar `Frame`s (y la capacidad de sus `Vec`) en vez de dropearlos → **–11/12%** más en llamadas, ~72 ns/llamada recuperados. El riesgo era de GC — un frame retirado retiene referencias stale — y se blindó con **una sola puerta de entrada** (`recycle`, que hace `scrub` incondicional), `debug_assert!(is_scrubbed())` en cada reuso **con la suite entera corrida en modo debug**, y la verificación de que el pool no está en ningún camino de raíces del colector

✓ **Éxito: logrado, y con creces**: un método que invoca en bucle ya no re-resuelve nada — pero la medición mostró que el mismo patrón estaba en **tres** opcodes, no solo en los invokes. Resultado: `getfield/putfield –86%`, `invokevirtual –45%`, `invokestatic –40%` (acumulando el pool de frames), sin cambiar una sola regla de la spec.

`F1` **Superinstrucciones / fusión** ☒ **salteado a propósito** Avanzado

Avanzado Queda disponible si en algún momento el piso de despacho vuelve a ser el cuello y hay instrumentación capaz de resolverlo

✓ **Éxito: decisión, no omisión**: fusionar `iload,iload,iadd` atacaría el piso de despacho (~28-30 ns/opcode) con una mejora estimada de 10-20% — **por debajo del ruido de layout de esta máquina (±3-12%)**. O sea: no podríamos *demostrar* que funciona. El esfuerzo se redirigió a `F3`, donde el efecto (2 órdenes de magnitud) es medible muy por encima del ruido.

F2 Inline caching + stack caching **pendiente** **Cumbre**

Cumbre **Inline cache monomórfico** (receptor → método). F0 ya dejó el lugar preparado: cachea el `vtable_slot` del tipo estático, y lo único que queda por llamada es `vtable_method(runtime_class, slot)` — que depende del receptor. `SiteKind::Vtable` está formado para alojarlo

Cumbre **Stack caching**: tope de la pila de operandos en registro. En el JIT es lo mismo que asignar registros a la pila de operandos (hoy cada operación Java es load/op/store contra L1)

✓ **Éxito**: dispatch dinámico acelerado en llamadas repetidas.

F3 JIT (bytecode → nativo) **primer tier corriendo** **Cumbre**

Cumbre **Emisor x86-64 propio** (`src/burst/x64.rs`), sin dependencias — las funciones de Windows se declaran a mano con `extern "system"`, igual que en su día se escribió el cursor de bytes en vez de usar `nom`. ALU, `idiv`, `jcc` con labels y parcheo de desplazamientos, ABI **Microsoft x64** (args en RCX/RDX/R8/R9, 32 bytes de shadow space, RSP alineado a 16). Verificado **ejecutando**: bucles, factorial, y un test de código generado **llamando a código generado**, que cubre de una sola vez las tres reglas del ABI que solo muerden en un `call`

Cumbre **Memoria W^X como garantía del tipo** (`exec_mem.rs`): `CodeBuf` (escribible) → `.make_executable()` **consume self** → `ExecMem` (ejecutable). Después de la transición no queda ningún handle escribible — no es que "recordamos no escribir", es que el tipo lo hace imposible

Cumbre **Compilador** (`compile.rs`) del subconjunto **frame-local de enteros** — el mismo que ya corría lock-free (W1), o sea sin heap, sin llamadas, sin referencias: un método así es una **función pura de sus locales**. Cualquier otro opcode ⇒ no se compila. La profundidad de la pila se recalcula **del CFG**, sin confiar en el `StackMapTable`: un desacuerdo entre ramas se rechaza en vez de generar código sobre una suposición

Cumbre **Las tres trampas de semántica**, con prueba: (1) *normalización* — todo `int` en registro es la sign-extensión de sus 32 bits; se emite `movsxd` tras la aritmética pero **no** tras `iand/ior/ixor`, con la demostración de por qué (los 33 bits altos de un sign-extendido son copias del bit 31); (2) *shifts* — Java enmascara a 5 bits, x86 a 6 en ops de 64, y `iushr` es lógico sobre 32 (`-1>>>1` debe dar `MAX_VALUE`); (3) *división* — `INT_MIN/-1` trunca como manda el JLS §15.17.2 en vez de fallar con `#DE`. Los 11 operadores binarios contrastados contra un modelo en Rust sobre **361 pares de valores de borde**

Cumbre **Deopt por pureza**: como el subconjunto no tiene efectos secundarios, ante cualquier situación que el nativo no pueda manejar (divisor cero) el código **abandona y el intérprete ejecuta el método desde cero** — re-ejecutar es indistinguible. Eso evita emitir manejo de excepciones en nativo, que es donde estos proyectos se vuelven inmanejables

Cumbre **OSR + poll de safepoint**, ambos restringidos a back-edges con **pila de operandos vacía** (un `goto` de retorno de `while/for` la tiene vacía → cubre los bucles reales), con lo que el estado a transferir se reduce a *locales + pc* en las dos direcciones. Sin OSR, un método con un bucle largo entrado **una sola vez** nunca calentaba: es exactamente el caso de `BmLoop`. El poll **sale hacia el intérprete** en vez de manejar el GC desde el nativo

Cumbre **El differential testing sale gratis**: con el JIT activo en `green/os-gil` y apagado en `os-parallel`, el oráculo `green==os-gil==os` que el proyecto usa hace meses **se convierte en la comparación JIT-vs-intérprete** sobre el corpus entero. La suite pasa igual con `JVM_JIT=0` y sin él. Además, un `static` estable para la palabra de poll: el `gc_pending` del driver es un `Arc` construido **fresco en cada corrida** (e inexistente en green), así que hornear su dirección habría sido corrupción silenciosa

Cumbre  **Pendiente, en orden de impacto**: ensanchar el subconjunto (hoy un `+= 256` descalifica un método entero por `wide iinc`; después `getstatic/putstatic` de ints, y llamadas), asignar **registros** a la pila de operandos, y habilitar el JIT en `os-parallel` — deliberadamente apagado, porque es el único modo con el heisenbug abierto y no se le agrega una variable a un problema que lleva ~17 sesiones de acotamiento

✓ **Éxito: la cumbre, alcanzada en su primer tier**: hay un compilador propio que emite x86-64 y lo ejecuta. `BmLoop` pasó de **475 ms a 4.1 ms — 116x** — y el número que lo explica no es el tiempo sino el conteo de opcodes: **17.325.011 → 620**. El bucle dejó de existir para el intérprete. Los cuatro workloads de control quedaron planos.

Fase G — `plain_data` / value types (modelo de datos)

Los “**huérfanos de Object**”: tipos planos sin header, sin identidad, sin monitor, flatteables. Viable porque tenemos compilador propio (Fase B) que puede emitir el constructo y una VM que lo trata especial. Es, en chiquito, el principio de representación de un **tensor** — puente directo a sistemas de ML. Inspiración: value classes de Valhalla, `struct` de .NET.

G0 `plain_data` plano en el heap Cumbre

Cumbre Tipo sin header: `[field0 | field1]` vs objeto normal `[class_ptr | fields]`

Cumbre El compilador (Fase B) lo marca; la VM lo guarda inline

✓ **Éxito:** un value type sin header conviviendo con los objetos normales.

G1 Arrays flatteados + semántica de valor Cumbre

Cumbre `plain_data[]` contiguo: sin los N punteros, ni los N headers, ni N alocações

Cumbre Igualdad por valor (comparar bytes), sin `null`, sin GC para ellos

✓ **Éxito:** comparar (medible) memoria/layout de `plain_data[]` vs un array de objetos.

G2 Escape analysis lite Cumbre

Cumbre Versión declarativa/estática (el automático completo es del JIT, Hito F3)

✓ **Éxito:** aplanar en *load time* cuando se prueba que un objeto no escapa.

Fase H — KajiLibrary (la biblioteca estándar, en Java, dogfooded)

La versión **concreta y accionable** de la Fase C: la biblioteca escrita a mano en `KajiLibrary/` (nuevo directorio en la raíz), **compilada por el `javac` propio** (dogfooding — cada clase que no compile es un ítem concreto de B6) y **ejecutada en KajiJDK**. Clasificada por *tiers* según su dependencia: ● Java puro (autocontenido, se compila y corre sin depender del runtime) · 🦋 necesita **intrínsecos del VM** (avanza con el track KajiJDK). El objetivo es la **Fase E** (cerrar el círculo): tu `.java` → tu `javac` → tus bibliotecas → tu JVM.

H1 KajiLibrary — la biblioteca estándar propia ✅ tiers 0–5 cerrados Cumbre

Base ✅ **Tier 0 — Núcleo / raíces** (mayormente 🦋 nativo) — `Object`, `Class`, `String`, `System`, la jerarquía `Throwable` → `Exception` / `RuntimeException` / `Error` + subclases, `Enum`, `Integer`, `Math`, `Thread`, `java.lang.ref/*`, `java.io.PrintStream`. Escrito **de cero** (el `bootstrap/` es solo **referencia**). **Hecho**: compila y pasa el gate; `String` implementa `Comparable` + `CharSequence`; #6 (`Object` `super_class`) arreglado → #0. Correr pende del runtime

Base ✅ **Tier 1 — Interfaces fundamentales** (● Java puro) — `Comparable`, `Comparator`, `CharSequence`, `Iterable`, `Iterator`, `Runnable`, `AutoCloseable`, `Cloneable`, `Appendable`. Triviales y **base de casi todo lo demás**. **Cerrado**

Base ✅ **Tier 2 — Wrappers + texto** (●) — `StringBuilder` (implementa `CharSequence`); `Number` + los 8 wrappers (`Integer` / `Long` / `Short` / `Byte` / `Character` / `Boolean` / `Float` / `Double`, todos `Comparable` con **bridge** `compareTo(Object)`). **Hecho** (`Void` incluido; correr el boxing/unboxing pende del runtime)

Avanzado ✅ **Tier 3 — Interfaces funcionales** `java.util.function` (●) — `Function` / `Consumer` / `Supplier` / `Predicate` con sus default de composición (`andThen` / `compose` / `identity`, `and` / `or` / `negate`). **Confirmado que el `javac` propio emite lambdas** (`invokedynamic` + `LambdaMetafactory`). `java.util.function` **completo**: + `BiFunction` / `BiConsumer` / `UnaryOperator` (extends `Function`) / `BinaryOperator` (extends `BiFunction`)

Avanzado ✅ **Tier 4 — Colecciones** `java.util` (●, gran superficie) — interfaces `Collection` / `List` / `Set` / `Queue` / `Map` (+ `Map.Entry`) e implementaciones `ArrayList` implements `List` / `HashMap` implements `Map` / `HashSet` implements `Set` con `iterator()` real (retrofit hecho: #9/#10 arreglados; el iterador va en clase same-file por #13). Faltan `Deque` / `LinkedList` / `ArrayDeque` (las utilidades `Objects` / `Optional` / `Arrays` / `Collections` pasaron a H2)

Cumbre ✅ **núcleo Tier 5 — I/O básica** (🦋 en los bordes) — framework `InputStream` / `OutputStream` / `Reader` / `Writer` + concretas Java-puro `ByteArray` / `String` / `BufferedReader` (dogfoodean `StringBuilder`). `PrintStream` real y `FileInputStream` / `OutputStream` arrastran intrínsecos del VM → track KajiJDK (runtime)

✓ **Éxito**: un programa real —con `StringBuilder`, colecciones y lambdas sobre sus interfaces funcionales— se **compila con el `javac` propio** y **corre en KajiJDK**, sin tocar el JDK de Temurin. La biblioteca se escribe a mano en `KajiLibrary/` (nuevo directorio en la raíz), en *tiers* por dependencia: ● Java puro (autocontenido, se dogfoodea entero) · 🦋 necesita **intrínsecos del VM** (avanza con el track KajiJDK/runtime).

H2 Utilidades y conveniencia ('`java.util`') 🦋 en curso Cumbre

Avanzado ✅ `java.util.Objects` — utilidad estática (`requireNonNull`, `equals`, `hashCode`, `isNull` / `nonNull`, `toString`). Hecho (8 miembros). Pero cablear su `requireNonNull` en los `default` de Tier 3 **espera el fix de #11**: el `javac` no resuelve llamadas estáticas a `java.util` (ni `Objects` / `Arrays` / `Collections`)

Avanzado ✅ `java.util.Optional<T>` — contenedor sin-`null` (`of` / `empty` / `ofNullable`, `get` / `isPresent` / `isEmpty` / `orElse` / `orElseGet`, `map` / `filter` / `ifPresent`). Dogfoodeó genéricos + inferencia + lambdas sobre las interfaces funcionales. Sumó `NoSuchElementException`

Avanzado ✅ `java.util.Arrays` — 20 miembros: `toString` / `equals` / `fill` / `copyOf` / `hashCode` sobre `int` / `char` / `boolean` / `Object[]` + `sort` (insertion). Escrita; **consumirla** desde otras clases espera #11

Cumbre ✅ `java.util.Collections` — `swap` / `reverse` / `fill` / `sort` ×2 (natural con type param acotado + por `Comparator`), sobre acceso indexado de `List`. Escrita; **consumo** espera #11 y que los concretos sean `List` (#9). Faltan `emptyList` / `unmodifiable*` / `binarySearch` (necesitan backing concreto o iteración)

Avanzado 🦋 **Stragglers de tiers previos** — ✅ `Void` (T2) y `BiFunction` / `BiConsumer` / `UnaryOperator` / `BinaryOperator` (T3 → `java.util.function` **completo**); falta `Appendable` (T1)

✓ **Éxito**: el esqueleto de tiers 0–5 se vuelve una biblioteca **usable**: las clases-pegamento que aparecen en casi todo el código real (utilidades estáticas y contenedores), todas ● **Java puro** y **desbloqueadas ya** (no dependen de los gaps del compilador #4–#10).

H3 java.util.stream — pipelines funcionales **núcleo (T1–T5)** **Avanzado**  **T1 — Substrato funcional** — las 6 especializaciones primitivas de `java.util.function``(IntFunction / ToIntFunction / IntPredicate / IntConsumer / IntUnaryOperator / IntBinaryOperator)` + la interfaz `Collector<T,A,R>`**Avanzado**  **T2 — Stream<T> core (eager)** — intermedias `filter / map / flatMap / distinct / sorted / limit / skip / peek + mapToInt / mapToLong / mapToDouble`; terminales `forEach / reduce / count / toList / toArray / anyMatch / allMatch / noneMatch / findFirst / min / max`; factories `of / empty . toList()` devuelve un `ArrayList` real. `collect()` + `Collection.stream()` esperan **#17****Avanzado**  **núcleo T3 — Collectors** — `toList / toSet / joining / counting` (+ el `Collector` concreto `CollectorImpl`), `gate-clean`. El `collect()` que los consume espera **#17** (override de variable de tipo a nivel método). Faltan `groupingBy / mapping`**Avanzado**  **T4 — Streams primitivos** — `IntStream / LongStream / DoubleStream` eager`(range / of / sum / average / map / filter / reduce / sorted / limit / skip / ... + boxed())` con `OptionalInt / OptionalLong / OptionalDouble . mapToObj` diferido (#7 / array genérico)**Cumbre**  **núcleo T5 — Avanzado** — `flatMap` (eager) + bridges primitivo→objeto (`Stream.mapToInt / mapToLong / mapToDouble , *Stream.boxed()`). **Fuera de alcance del modelo eager** (piden rediseño lazy): `Splititerator` / lazy real, `iterate / generate` infinitos, paralelismo verdadero

✓ **Éxito:** el pago de Tier 3 (interfaces funcionales) + Tier 4 (colecciones): `coleccion.stream().filter(...).map(...).collect(...)`. Dogfoodea el compilador como nada — lambdas encadenadas, genéricos, method refs. **Eager primero** (cada op intermedia materializa en un backing interno; la terminal consume): correcto para streams finitos y mucho más simple que el lazy; el lazy real (`Splititerator` + pipeline diferido) es un tier avanzado. Se apoyó en los **retrofits de #9** (ya hechos: `ArrayList / HashMap / HashSet` son `List / Map / Set` reales con `iterator()`), así que `toList()` devuelve un `ArrayList` de verdad. Núcleo T1–T5 hecho; solo `collect()` + `Collection.stream()` esperan **#17** (override de variable de tipo a nivel método).

H4 java.time — fecha y hora como value types **núcleo (T1–T5)** **Cumbre**  **T1 — Abstracción** `java.time.temporal` — 7 interfaces`(TemporalAccessor / Temporal / TemporalField / TemporalUnit / TemporalAmount / TemporalQuery / TemporalAdjuster)` + enums `ChronoField / ChronoUnit`. El substrato sobre el que se montan los value types**Cumbre**  **T2 — Duración e instante** — `Duration` (seg+nanos, `TemporalAmount`) e `Instant` (epoch seg+nano, `Temporal`, `now()` 🐦 = `System.currentTimeMillis` native). Normalización de nanos con floor manual**Cumbre**  **T3 — Fecha local** — `LocalDate`: la conversión **epoch-day** ↔ (**año,mes,día**) del JDK (bisiestos, largos de mes) compilada y andando; `plusDays / plusMonths / getDayOfWeek` + enums `Month / DayOfWeek`**Cumbre**  **T4 — Hora y fecha-hora** — `LocalTime` (h/m/s/nano, wraparound de día) y `LocalDateTime` (compone T3+T4, la aritmética de horas carga días en la fecha)**Cumbre**  **núcleo T5 — Períodos y parciales** — `Period` (`TemporalAmount`), `Year / YearMonth` (`Temporal`), `MonthDay` (`TemporalAccessor`). **Diferido (backlog):** zonas (`ZoneId / ZonedDateTime / OffsetDateTime`, piden datos tz) y `DateTimeFormatter` con patrones

✓ **Éxito:** el dogfood de las **clases inmutables**: cada tipo es `final` con campos `final`, y `plus / minus / with` devuelven instancias nuevas. Casi todo es aritmética pura — el único seam 🐦 es el reloj (`now()` → un `System.currentTimeMillis()` native). **Solo calendario ISO-8601, sin zonas** (los `Local* + Instant / Duration / Period` son zone-free; `ZonedDateTime` y afines a T5). **Con la abstracción `java.time.temporal`** — más fiel al JDK: jerarquía de interfaces + enums que implementan interfaces + switch sobre enum. **Núcleo T1–T5 hecho: 20 clases, 273 match, gate PASS** (30 clases con java.time completo). `TemporalAccessor.query / range` omitidos del subset (default genérico heredado no se puede llamar, familia #15). Enums en paquete nombrado necesitaron ctor explícito (**#18**).

H5 java.util.regex — motor de expresiones regulares **T1–T4 + T5 parcial** **Cumbre**  **T1 — API pública** — `Pattern` (`compile / matcher / pattern / quote / matches`) + `Matcher``(matches / find / lookingAt / group / start / end / groupCount / reset)` + `PatternSyntaxException`. La superficie sobre la que se apoya todo**Cumbre**  **T2 — Parser** — regex → árbol de nodos: literales, `.`, clases `[...]` (rangos, negación), cuantificadores greedy `*/+/?/{n,m}`, grupos `(...)`, alternación `|`, anclas `^/$`, escapes `\d/\w/\s` y sus negaciones**Cumbre**  **T3 — Motor de matching** — backtracking recursivo continuation-passing; `matches() / find() / lookingAt()`; `Quantifier` greedy con loop-back + `Prolog` (reset de contador para cuantificadores anidados) y guard de zero-width**Cumbre**  **T4 — Grupos y reemplazo** — grupos capturadores → `group(n) / start(n) / end(n)` (con guardar/restaurar en backtrack); `replaceAll / replaceFirst / quoteReplacement` con referencias `$n`**Cumbre**  **T5 (parcial) — Avanzado** — HECHO y validado: reluctant `?/+/{}?`, grupos non-capturing `(?:...)`, lookahead `(?=...)/(?!...)`, backreferences `\1–\9`. **Backlog:** possessive `+`, lookbehind `(?<=...)`, grupos con nombre `(?<n>...)`, clases Unicode/POSIX `\p{...}`, flags DOTALL/MULTILINE

✓ **Éxito:** un motor de regex de verdad, 100% Java puro — el dogfood más algorítmico de la fase: un **parser** de la sintaxis regex + una **máquina de matching por backtracking recursivo** sobre una **jerarquía de nodos con match polimórfico** (cada nodo hace `match(m, pos, seq)` y, al tener éxito, llama al nodo siguiente — *continuation-passing*, lo que habilita backtrackear en los cuantificadores). NFA con backtracking (no DFA), char-based. **Núcleo cerrado y VALIDADO:** como el track de biblioteca no ejecuta, se portó el mismo algoritmo a Java puro y se corrió con el JDK contra `java.util.regex` como oráculo → **138/138 checks** (`matches` + `find-all` con spans por grupo + reemplazos). Cero findings de correctitud del compilador. 18 clases (`Pattern/Matcher/PatternSyntaxException` + jerarquía de nodos + parser).

H6 java.util.Formatter / String.format — formato printf **diseñado** **Cumbre**

 **T1 — API + parser + conversiones base** — `java.util.Formatter` (`format` / `toString` / `out` / `flush` / `close` , `Closeable` / `Flushable`) + `String.format` ; parser del format string → chunks literales + specifiers; conversiones `%s` / `%S` , `%%` , `%n` , `%b` , `%c`

 **T2 — Enteros** — `%d` / `%x` / `%X` / `%o` (radix vía `Integer` / `Long`), con flags `-` (left-justify) / `0` (zero-pad) / `+` / espacio / `(` (negativos) / `,` (grouping) y width

 **T3 — Punto flotante** — `%f` / `%e` / `%E` / `%g` / `%G` con precisión (default 6), redondeo, width y flags. La parte algorítmica dura: `double` → string decimal

 **T4 — Flags/width/precision + índice de argumento** — pulido transversal: `#` (alt-form), precisión en `%s` , índice `%n$` y relativo `%<` , edge cases de width/precision por conversión

 **T5 — Avanzado (diferido)** — `Locale` , conversiones de fecha/hora `%t` , `%a` (hex-float), `Formattable` , e integración con `java.text` (`DecimalFormat` / `NumberFormat` / `MessageFormat`)

✓ **Éxito:** el formato de texto estilo printf, 100% Java puro — mismo espíritu que regex: un **parser** de un mini-lenguaje (el format string `%[arg$][flags][width][.prec]conv`) + un **motor de conversión** que formatea cada argumento según su especificador. `Formatter` envuelve un `Appendable` y `String.format` es el atajo. Se valida con el mismo truco del self-test (portar el algoritmo al JDK y comparar contra `java.util.Formatter`).

H7 Fuera de alcance por ahora **backlog** **Cumbre**

 **java.nio** — I/O de canales y buffers ( descriptors de archivo → runtime)

 **java.util.concurrent** — concurrencia de alto nivel ( threading / modelo de memoria → runtime; su **núcleo ya vive en el track de la JVM** —AQS, locks, `Executor` , `ConcurrentHashMap` , `CompletableFuture` —, ver la sección «Concurrencia»)

 **reflexión completa** — `Class` / `Field` / `Method` / `Constructor` ( metadata del VM → runtime)

✓ **Éxito:** lo que queda para **acotar** el scope, todo **acoplado al runtime** (espera el track KajJDK): I/O de bajo nivel, concurrencia y reflexión. `java.text` (formato por patrón) es autocontenido y vive como el T5 diferido de H6.

Estado actual

Fase A / Hitos A0–A5 logrados; A6 en progreso; A7 (conformidad JVMs) con sus 8 accionables cerrados. Fase B / front-end + semántica + chequeo + flujo + desugar + Codegen completo (StackMapTable e `invokedynamic`), con B5 cerrado y B6 (fidelidad de `javac`) en curso. Fase H / KajiLibrary: tiers 0–5 cerrados y `stream / time / regex` con su núcleo. El proyecto compila **sin warnings**, pasa sus tests en verde (142 del runtime en esta línea de trabajo; 532 contando los del compilador propio), cubre **el set de opcodes completo** (199 de 199 alcanzables, `invokedynamic` incluido), produce **fixtures byte-idénticos** a `javap` y ya ejecuta **bytecode real con objetos, GC generacional, hilos (green y de SO bajo un GIL), monitores y un sistema de tipos completo**: aritmética, control de flujo, recursión, `switch (tableswitch / lookupswitch)`, un **heap** con objetos/herencia/campos/arrays, **dispatch dinámico**, **excepciones**, **inicialización de clases**, **class loaders**, un **recolector generacional** (young por copia + Old mark-sweep-compact, con **referencias débiles** de `java.lang.ref`), **green threads** (scheduler cooperativo) con **monitores** (`synchronized` de bloques y métodos, `wait / notify`), y un **verificador de bytecode JVMs-estricto**.

- **ClassReader** (cursor big-endian) y **constant pool** completo (`ConstantPoolEntry`, 17 tags + `Tombstone`), con árbol de referencias en `pretty_class_visualizer.rs`.
- **Header**: versiones, `access_flags` (+ métodos `is_*`), `this_class / super_class` con `class_name().from_path() → Result` (sin panics).
- **Code** con **desensamblado completo** (opcodes + `tableswitch / lookupswitch / wide`) y comentarios `// ...` resueltos.
- **StackMapTable** (los 7 frame types), cada frame en su clase bajo `parser/stack_map_table/`, con `verification_type_info`.
- `javap` **brief y -v byte-idénticos** (incl. cabecera Classfile / Last modified / SHA-256). Flags de visibilidad (`-public / -protected / -package / -p`).
- Renderers factorizados en `parser/printers/`: `verbose` (orquestación) + `file_header` + `pool_comments` + `member_dump` + `brief` + `dump_common` + `visibility`.

Pendiente de A0 (no esencial, no bloquea avanzar): atributos `Signature` (reescribe la declaración → parser de firmas genéricas), `BootstrapMethods`, `InnerClasses / EnclosingMethod`, `Exceptions`, `ConstantValue`, anotaciones, `Record`, `PermittedSubclasses`, `NestHost / NestMembers`, `LocalVariableTable / Type`; y flags de contenido (`-c / -l / -s`).

A1–A2 — el intérprete. Loop de despacho sobre una **pila de frames**; cada frame referencia su bytecode por índice (`MethodId`) en el `Method Area` (`metaspace`, con resolución de llamadas por el código del constant pool). Opcodes: `iconst / iload / istore`, `iadd / isub / imul`, `goto / if_icmpgt`, `invokestatic / ireturn`. Corre **factorial** y **fibonacci recursivos**. Visualizador `jvm-step` paso a paso con vista de dos paneles de la call stack.

A3 — objetos y heap. El **heap** es una arena de bytes (`Vec<u8>`) con `malloc` bump (sin liberar; el GC llega en A5). Los objetos se disponen como `[class_id | mark | campos]` con `layout` que respeta la herencia; cada clase recibe un **Class ID** (UUID) y un *mirror* en el heap para sus estáticos (estilo HotSpot). Opcodes: `new`, `dup`, `getfield / putfield`, `aload / astore`, los cuatro `invoke*` y la familia de arrays (`newarray / anewarray`, `arraylength`, `iaload / iastore`, `aaload / aastore` + `byte/char/short`). El **dispatch dinámico** usa `vtables` por clase (slot del tipo estático, método del tipo real) e `itable` para interfaces — verificado: `Animal a = new Dog(); a.sound() → Dog.sound`.

A4 — robustez del runtime. Excepciones: `athrow` + tabla de excepciones + `stack unwinding` por la pila de frames, con `finally` (catch-all + re-throw) y las **implícitas** que la VM sintetiza (NPE, índice fuera de rango, cast inválido, tamaño negativo). **Verificador** de bytecode (type-checking en una pasada con el `StackMapTable`) que corre antes de ejecutar. **Inicialización** de clases `<clinit>` perezosa y super-primero. **Class loaders** bootstrap/app con delegación parent-first (las `java.lang.*` propias viven en `boot/`). Resolución que falla con **errores de linkage** (`NoClassDefFoundError / NoSuchMethodError`) en vez de paniquear. **Robustez fina cerrada (2026-08):** `Throwable` con `getMessage / toString` y **stack traces** capturados por la VM en el throw; `ExceptionInInitializerError` (JVMs §5.5: un `<clinit>` que lanza propaga, se envuelve, y deja la clase *erroneous* → `NoClassDefFoundError` al reuso); **default methods** de interfaces (JSR 335 — la `vtable` fusiona superinterfaces, regla *maximally-specific*). Pendiente fino: chequeos de acceso e identidad (`nombre`, `loader`) (no producibles con `javac` — ver `holdon.md`).

A5 — nativos + GC. Puente nativo (`System.out.println` imprime de verdad) e *intrinsic*s (`getClass`, `hashCode`, `Math`, `arraycopy`, `String / Class`).

Recolector de basura que arrancó en mark & sweep y creció a un colector con **compactación** (mover objetos + reescribir punteros), **free list** con *coalescing*, **política de fragmentación** configurable, **disparadores** (out-of-space / occupancy / allocation-rate / explícito) sobre un *safepoint*, y **marcado transitivo** correcto (sigue campos, arrays y estáticos).

A6 (en progreso) — cumbre del runtime. Verificador de bytecode JVMs-estricto: además de la cobertura completa de opcodes (incluido `switch`), verifica **con o sin StackMapTable** — inferencia por punto fijo como fallback, con *cross-check* de la tabla contra la inferencia — sobre un **gate estructural** (§4.9), con **tipos de locales** estrictos + `max_stack`, reglas de **objetos sin inicializar**, **handlers de excepción** tipados y **acceso/linkage** (regla `protected`, `count` de `invokeinterface`). **Sistema de tipos completo:** los cuatro tipos numéricos (`int / long / double / float`) **ejecutados y verificados** — cómputo, conversiones, comparaciones, división con `ArithmeticException`, **categoría-2** (params, campos, estáticos, arrays, frames del `StackMapTable`) y el **lattice de referencias** (covarianza de arrays + *join/LUB*). **GC generacional:** arena dividida en `Eden | S0 | S1 | Old`; los objetos nacen en Eden, un **minor** por **copia** evacúa los pocos sobrevivientes a un *survivor* (o los **promueve** a Old por edad) reescribiendo referencias con *forwarding*, un **write barrier** mantiene un **remembered set** de punteros `old→young` (las raíces que el minor escanea, sin recorrer todo Old), y el **major** hace mark-sweep-compact sobre Old. **Referencias débiles** (`java.lang.ref`): `WeakReference` + `ReferenceQueue` — el major trata el *referent* como débil y, al morir, lo limpia (`get() → null`) y **encola** la referencia; cimienta de `PhantomReference / Cleaner`.

Cobertura del set de opcodes — 199/199 alcanzables. El intérprete despacha todos los opcodes del JVMs que un class file moderno puede contener. Los últimos en cerrarse fueron `nop` (0x00), `goto_w` (0xc8) — el `goto` de offset de 4 bytes, para targets a más de ±32 KB, implementado ensanchando `jump_to` hacia un `jump_to_wide` común en vez de duplicar la aritmética — y el prefijo `wide` (0xc4), que reejecuta el opcode siguiente con un índice de local de **16 bits**: la única manera de direccionar los slots pasados el 255 en un método con más de 256 locales (`wide iinc` mide 6 bytes, el resto 4). Ese último salió barato por una propiedad que el diseño ya tenía: los handlers de `variable_operations` reciben `slot: usize` y **nunca se enteran del ancho**, así que ensanchar es puro *decoding* y el módulo no cambió de comportamiento. Verificado por *differential testing*: `WideLocals.java` declara 300 locales para forzar a `javac` a emitir el prefijo (`istore_w 299`, `iinc_w 299, 35`, `iload_w 299`), y el mismo `.class` da 42 tanto en el `java` de JDK 25 como en nuestra VM. **De los 4 restantes, 3 son una decisión y no deuda:** `jsr / ret / jsr_w` están prohibidos por **JVMs §4.9.1** en class files de versión 50.0+ (Java 6 en adelante) — desde que existe el `StackMapTable` y el verificador por *type-checking*, las subrutinas quedaron fuera del modelo, y `javac` compila *finally* duplicando el

bloque. La postura del proyecto es **leer sí, ejecutar no**: el desensamblador soporta `jsr / ret / jsr_w / ret_w` porque un `javap` debe renderizar *cualquier* class file legal (lo exige A0, medido sobre `java.base`), mientras el gate estructural del verificador los rechaza y el intérprete no los despacha. Curiosamente *ejecutarlos* sería lo más fácil que queda —unas 10 líneas—; el costo está en una variante `ReturnAddress` que toca todos los `match` sobre `Value` y, sobre todo, en verificar subrutinas polimórficas, la parte más difícil del type-checker del JVMs. Razonamiento completo en la nota de diseño `subrutinas-jsr-ret.md`. Contarlos como faltantes mezclaría una decisión con una tarea: el número honesto es **199 de 199 alcanzables — completo**.

invokedynamic (0xb) — el subsistema. El opcode solo no hace nada: no nombra ningún método, sino un *bootstrap* que **produce** el destino la primera vez. Esa indirección es lo que le permitió a Java agregar lambdas, concatenación de strings y records sin inventar un opcode por cada una — el lenguaje pone la política en el bootstrap y la VM queda fija. Corren **5 de las 6 fábricas** que emite `javac`: `StringConcatFactory`, `SwitchBootstraps.typeSwitch` (patrones de tipo y de enum), `ObjectMethods` (los tres métodos de un `record`, desde una sola entrada de `BootstrapMethods` y distinguidos por el *nombre* del call site), `LambdaMetafactory.metafactory` y `ConstantBootstraps.invoke`. La sexta, `altMetafactory`, necesita serialización (`java.io`) y queda fuera de alcance. Dos decisiones de diseño: los *bootstrap methods* se modelan como **intrínsecos** —el mismo criterio de `intrinsicsec.md`—, con `java.lang.constant` escrito en **Java**; y `LambdaMetafactory` **genera una clase real** al vuelo (con el escritor de `.class`): implementa la interfaz funcional, con campos para las capturas y un método SAM que reenvía al handle — igual que el JDK, que la spinnea con ASM. El objeto lambda es entonces una instancia común, despachada por el itable normal, sin atajo. Ruta, correcciones y mediciones: `invokedynamic-ruta.md`.

La VM puede invocar Java (call_java). Empuja un frame propio y lo corre con un bucle de `run_one` anidado, devolviendo el resultado. Resultó ser el **caso general de lo que la inicialización de clases venía haciendo**: `<clinit>` era la variante sin argumentos ni resultado, así que el mecanismo ya existía entero y sólo estaba especializado. Importa porque **los intrínsecos dejan de ser terminales**: un nativo que sólo puede calcular y devolver está obligado a reimplementar lo que necesite de la biblioteca — así fue como `String.valueOf` terminó a medias en Rust. Con esto, `String.valueOf(Object)` llama al `toString()` del objeto, un `record` pregunta el `equals / hashCode` de sus componentes en vez de comparar referencias, y una constante dinámica ejecuta su bootstrap. Eso **cerró 0xb**: el **modelo de objetos de `java.lang.invoke`** (`MethodHandle / MethodType / Lookup`) vive en Java, con `MethodHandle.invoke / invokeWithArguments` nativos —polimorfía de firma, interceptados antes de la resolución—, el único intrínseco que corresponde. El premio se cobró: `ConstantBootstraps.invoke` es **ahora Java** (`handle.invokeWithArguments(args)`), lo que obligó a hacer el **cache de condy raíz de GC**. El `ldc` de `MethodHandle / MethodType` —que `javac` nunca emite— se probó con **class files hechos a mano por el escritor de `.class`** (su estreno). Con mirrors de primitivos (`int.class` → `Integer.TYPE`) y los kinds static/virtual/special/constructor, y `LambdaMetafactory` generando clases reales, `invokedynamic` queda completo.

multianewarray (0xc5). El último en cerrarse, y el que más enseña sobre el modelo de datos: **Java no tiene arrays rectangulares**. `new int[2][3]` no es un bloque plano sino dos `[I` colgando de un `[[I`, cada nivel un objeto real — por eso las filas se reemplazan por separado y `a[0].length` no tiene por qué igualar `a[1].length`. La instrucción lleva el descriptor del array *mismo* (`[[I`, no el elemento como `anewarray`) más un contador de dimensiones, porque sólo se materializan **esos** niveles: `new int[3][ ]` deja los internos en `null`. Los conteos se validan *todos* antes de allocar nada, para que un largo negativo en una dimensión posterior no deje un array a medio construir; los niveles superiores guardan referencias y sólo el más interno usa el ancho real del elemento (una fila `byte[ ]` ocupa un byte por elemento, no cuatro); y las referencias hijo se escriben por `store_reference`, nunca por un `write_u32` crudo, porque son exactamente los punteros `old+young` que el *remembered set* debe registrar. **Ejecutarlo era la mitad del trabajo**: hubo que modelarlo en el `transfer` del verificador, ya que un opcode sin modelar produce un `VerifyError` marcado `unsupported` — que por diseño hace que el llamador avise y siga, o sea que la clase habría corrido con el verificador claudicando en silencio.

Green threads (Fase 0). `java.lang.Thread + Thread.start()` nativo, un scheduler **cooperativo** round-robin que conmuta en el safepoint (entre opcodes), un call stack por-thread, y el GC tomando las raíces de **todos** los threads. Verificado: dos workers intercalándose con `main` (que los espera por spin-wait) dan `100`, y el step-visualizer muestra los cambios de contexto, el spawn y la terminación de cada thread. Es el modelo de los *virtual threads* de Java 21 (scheduling en espacio de usuario), con un solo carrier.

Hilos de SO + GIL (E1+E2). El substrato es un **parámetro de aplicación** (`JVM_THREADS`, como los `JVM_GC_*`): en modo `os-gil` cada `Thread.start()` lanza un `std::thread` real y la VM entera vive tras un **GIL** (`Arc<Mutex<JVM>>`) que se toma y se suelta **por opcode**, en el mismo safepoint que ya usaba el GC. El bloqueo es `park / unpark` de verdad (centralizado en `make_runnable`) para monitores, `wait / notify` y `join`, y `Thread.sleep` pasa a tiempo real. El GC corre seguro porque el GIL es el stop-the-world: un solo hilo toca la VM a la vez, así que heap, monitores y metaspace quedan correctos sin sincronización extra. Es el camino canónico — el de CPython y las JVM tempranas —: **correcto pero todavía serializado**. Validado con 4 tests de modo OS que dan el mismo resultado que en green, usando el intérprete cooperativo como oráculo de corrección. **El costo real de este salto no fue el JIT sino el refactor de ownership**, y lo abarató el seam: toda escritura de referencia ya pasaba por `HeapService.store_reference`.

Monitores. Cada objeto tiene su monitor intrínseco (lock reentrante con dueño + *blocked-set* + *wait-set*). `synchronized` funciona en sus dos formas: **bloques** (opcodes `monitorenter / monitorexit`) y **métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma el monitor del receptor o del `Class` al empujar el frame y lo suelta al popearlo, por `return` o por *unwind* de excepción, vía un único punto de release imposible de saltar). La señalización `wait / notify / notifyAll` libera el monitor (guardando el conteo de reentrada), parquea al hilo en el *wait-set* y lo re-adquiere al despertar. Verificado con exclusión mutua real (dos hilos × 100 incrementos → 200, contra el control sin lock que pierde updates) y el *handshake* productor/consumidor (`wait` → `notify` → 42). Después se cerró el resto: `IllegalMonitorStateException` (gate `owns_monitor` sobre `monitorexit / wait / notify`, JVMs §6.5), `Object.wait(long)` con **timeout** (deadline sobre `sleep_until`; reloj de opcodes en green, tiempo real en OS — demo `WaitTimeout` → 7) y el **monitor GC-safe**: los monitores se keyean por offset del heap, así que al mover objetos (minor y compact) las claves se remapean por el *forward* del GC, y los monitores de objetos colectados se descartan — ya se puede compactar con monitores tomados (demo `GcMonitor` → 5, y el test se cuelga si se quita el remapeo).

Pendiente del monitor: la interrupción del `wait` (llega con H1), biased/thin locks y detección de deadlock.

Arquitectura en capas (estilo service). El heap se encapsuló tras un `HeapService` que es su **único dueño**: toda escritura de referencia pasa por un portón (`store_reference`) que aplica el **write barrier** de forma no-bypassable — el seam donde irá la sincronización cuando los threads pasen a ser de SO. El runtime es la **JVM** (composition root) sobre `HeapService + MetaspaceService` + el sistema de threads.

Fase B — el compilador (front-end + semántica + chequeo + flujo + desugar). El `javac` propio (crate **desacoplado** en `src/javac/`, cuyo único contrato es el formato `.class`) tiene el front-end entero, las **dos pasadas** semánticas, el análisis de flujo y el desugar. **Front-end: lexer** (las 115 `TokenKind` de JDK 25, *maximal munch*, BOM), **parser** por *recursive descent* con **todas las sentencias** (incl. `switch` con patterns/guards, `try` -with-resources, `synchronized`), `enum / record / @interface` y **genéricos completos** — argumentos de tipo, *wildcards*, cotas y el diamante, partiendo `>> / >>>` con un *undo log* para no corromper el *backtracking*. El **lenguaje moderno** también entró: **lambdas** (con el desambiguador `(` -cast-vs-paréntesis-vs-parámetros por *lookahead* de la flecha), **referencias a método** (incl. `T[ ]::new`), **clases anónimas y locales**, el **type witness** (`Collections.<String>emptyList()`), que además

se **aplica**—fija los parámetros de tipo en vez de inferirlos—) y las **anotaciones** (§9.7), que antes se **descartaban** y ahora se cuelgan de la declaración. Cada forma se parsea entera y se guarda fiel en el AST; compilarlas es de fases posteriores, y hasta entonces la barrera del emisor las corta. **Pasada 1 (Enter/MemberEnter)**: tabla de símbolos con paquetes y tipos **anidados**, detección de duplicados y **ciclos de herencia**, un **class finder real** que lee `.class` del classpath (lector propio) **incluido el atributo `Signature`** — así `java.util.List` carga con su `<E>` y `List.get` devuelve `E`, no `Object` — resolución con **errores posicionados**, y el **grafo tipado persistido** como salida. **Pasada 2 (Attribute)**: resuelve nombres y tipa los cuerpos **decorando el AST in situ** (tipo + *binding* por nodo, slot por local con categoría-2, al estilo `JCTree.type / sym`); con **overload resolution en 3 fases** (estricta → boxing → varargs, con *most specific* y ambigüedad) y **genéricos** reales — subtipado con *containment* de wildcards (invariancia), sustitución del receptor, `lub / glb` e **inferencia** de los argumentos de tipo de una llamada (Cap. 18). El álgebra de tipos vive en un módulo `types` (el `Types` de javac) y la inferencia en `infer` (el `Infer`). **Pasada Flow (flow)**: sobre el AST decorado, la **asignación definitiva** del Cap. 16 — con los dos conjuntos duales *definitely assigned / definitely unassigned*, los flujos *when-true/when-false* de `&& / || / !`, las reglas de variables `final` (no reasignar, asignación única, multi-catch implícito), la **alcanzabilidad** (§14.21) y el `break / continue` **etiquetados** (§14.15/§14.16: la pila de contextos de `break` lleva la etiqueta a la que responde cada uno). **Pasada Desugar (desugar)**: baja el azúcar de **sentencias** (`for-each` → `for` indexado o `while` + `Iterator`, `try-with-resources` → `try/finally` + `close()`, `assert` → `if(!cond) throw`) y de **expresiones** (concatenación de `String` → cadena de `StringBuilder`, asignación compuesta `x op= y` → `x = (T)(x op y)` con el cast de reducción, y **varargs** → array en el *call site*) — verificado re-tipando el árbol bajado. En esta iteración se sumaron cuatro *rewrites* más: (1) `++ / --` **en posición de descarte** (§15.14.2/§15.15.2) — como sentencia (`x++`; `--x`;) o en el `update` de un `for` — se reescriben a `x += 1` reusando el *lowering* de la asignación compuesta (mismo cast de reducción), mientras que en posición de *valor* (`y = x++`) quedan para el codegen (`iinc / dup`); (2) la **switch-expresión** en posición de cola (§15.28) — `T v = switch..., return switch..., x = switch..., yield switch...` — se convierte en una **switch-sentencia** cuyos brazos **asignan** el valor (con temporal cuando hace falta); requiere `default` y etiquetas constantes, y los brazos con **bloque** o de **dos puntos** con `yield` (incluso adentro de un bucle o un `if`) se bajan reescribiendo cada `yield e` a `{ v = e; break $sw; }` con un `break` **etiquetado** sobre el `switch` generado —un `break` pelado solo saldría del bucle— mientras que el exhaustivo sin `default` y las switch-expresiones embebidas quedan como expresión sobre la pila; (3) **try-with-resources con excepción suprimida** (§14.20.3.1) — corrección de un bug: antes la excepción del `close()` *tapaba* la del cuerpo; ahora se preserva la primaria y la del `close()` queda **suprimida** (`Throwable.addSuppressed`), con la traducción completa del JLS (`$primary=null; try{...}catch($t){$primary=$t;throw;}finally{ if(r!=null){ if($primary!=null) try{r.close();}catch($x){$primary.addSuppressed($x);} else r.close(); } };`); (4) **switch sobre String** (§14.11) → **dos** switches sobre `int`: el primero mapea `s.hashCode()` a un índice sintético vía `equals()` (cadena *if/else* para las colisiones de hash) y el segundo replica los brazos indexados (preservando *fall-through* y `default`), con el `String.hashCode()` calculado en tiempo de compilación (algoritmo del JDK: `31·h + c` sobre unidades UTF-16, aritmética `i32` que envuelve); y (5) **switch sobre enum** (§14.11) con el `$SwitchMap` **fiel a javac**: `switch(color) → switch($C$1.$SwitchMap$Color[color.ordinal()])`, donde `$SwitchMap$Color` es un `int[]` sintético alojado en una **clase anidada** `C$1` y poblado en un `<clinit>` con un `try { map[Color.C.ordinal()] = k; } catch (NoSuchFieldError e) {}` por constante (robusto ante recompilación separada del enum). Esto requirió una **máquina de síntesis a nivel clase**: el miembro `Member::StaticInit` (con lo que los bloques `static { }` del usuario, antes **descartados**, ahora se conservan y analizan), la tabla de símbolos **mutable** en el desugar (para registrar la clase y el campo sintético, que la re-atribución tiene que ver) y el `enum extends Enum` implícito (§8.9, de donde sale `ordinal()`), y (6) **switch con type patterns** (§14.11.1) → una cadena de `instanceof` dentro de un bloque **etiquetado**: cada brazo es un `if` **suelto** (no `else if`) para que una **guarda que falla** caiga al `case` siguiente, el binding se materializa con un cast (`T v = (T) $t`), porque el `instanceof` del AST no lleva variable, se sale con `break` etiquetado —omitido cuando el brazo ya retorna, para no generar código inalcanzable— y un selector `null` sin `case null` lanza **NPE**. Esto exigió además que **Attribute bindee** la variable de patrón (visible en la guarda y en el cuerpo) y que **Flow** la vea siempre asignada dentro del brazo, como la del `catch`. y (7) el **guard** `$assertionsDisabled` del `assert` (§14.10): la aserción pasa a `if (!$assertionsDisabled && !cond) throw ...`, con un campo sintético `static final boolean` y el `<clinit>` que lo calcula de `C.class.desiredAssertionStatus()` — es lo que hace que una aserción **no cueste nada** con las aserciones apagadas (el default), porque al ser `static final` el JIT lo pliega. Con eso la pasada **no tiene reescrituras pendientes**, y además **materializa los miembros implícitos de un record** (campos, constructor canónico y *accessors*): sus símbolos ya venían de `enter` —por eso `p.x()` resolvía— pero sin cuerpo en el AST el `.class` los referenciaba **sin declararlos**, y un `record` compilaba a una clase **vacía**. Y ya está la **síntesis equivalente del enum** (§8.9.3): las constantes como campos, el `$VALUES`, el constructor privado (`String, int`) —lo único que puede darle a `java.lang.Enum` su nombre y su ordinal— y `values() / valueOf()`. Destruiría obligó a cerrar antes la **invocación explícita de constructor** (`super(...)` / `this(...)`, §8.8.7.1), que **no existía** —el parser la producía y la atribución la rechazaba, así que no se podía heredar de ninguna clase con constructor parametrizado— y a sumar `java.lang.Enum` a `bootstrap/ + boot/`, sin la cual el `.class` emitido no se verificaba ni corría. En el camino salieron a la luz cuatro fugas **silenciosas** del back-end, ya cerradas: el emisor **nunca generaba** `<clinit>` (el `$VALUES`, el `$SwitchMap` y el `guard` del `assert` quedaban declarados y en cero), los **inicializadores de campo** (`int v = 7;`) **no llegaban al .class**, `a.length` se tipaba `Unresolved` —y `i < a.length` salía comparando con `if_acmpne`— y un `new` de un tipo sin resolver no emitía nada. Falta el `enum` con constantes **parametrizadas** (`ROJO("rojo")`) o constructor propio; el boxing/erasure queda para `TransTypes`. Desde entonces el desugar ganó tres reescrituras grandes: **LambdaToMethod** (lambdas → método sintético `lambda$... + invokedynamic` con `LambdaMetafactory`, capturando locales y `this`), las **referencias a método** y el `record` por `ObjectMethods` (ambos por un nodo `Indy` genérico), y la **captura de this\$0** de las clases internas de instancia. Un binario `javac-step` recorre las cuatro fases **paso a paso** (tokens → AST → tabla de símbolos → chequeo), el modo `--desugar` muestra el árbol bajado, y la semántica está citada a la JLS 25 en `docs/pasada2-attribute.md` y `docs/semantica-jdk25.md`.

Fase B — Codegen (B3): objetos y flujo de control. El `compile()` corre ya las **pasadas** en el orden de javac (`Enter` → `Attribute` → `Check` → `Flow` → `Desugar` → `re-Attribute` → `Codegen`): **Flow antes que Desugar**, para que sus errores apunten al código fuente y no al bajado, y una **re-atribución** después, porque las reescrituras dejan nodos frescos sin `ty / binding / slot` y el emisor los necesita para elegir el opcode. Un programa con errores semánticos **no emite** `.class`. El **escritor** (`class_writer`) es propio y desacoplado: constant pool con deduplicación —incluidas las entradas `String`, `Long`, `Double` y `Float`, con el hueco que la JVMs exige tras cada `Long / Double` (ocupan dos entradas, y si no se reserva se corren todos los índices)— más la sección de **campos** y los atributos `Code / SourceFile`. El **emisor** cubre hoy: literales de todos los tipos (`ldc_w` para los anchos), la **promoción numérica binaria** (§5.6.2) con los doce opcodes `x2y` —`longA + 1` inserta el `i2l`, y los desplazamientos quedan exceptuados (§15.19)—, **objetos** (`new + dup + invokespecial`, `getField / putField`, `getStatic / putStatic`, `invokeVirtual`, `checkCast`), **asignación** a local y a campo, y **flujo de control** completo (`if / else`, `while`, `do-while`, `for`, `break`, `continue`) con etiquetas y parcheo de saltos, comparaciones por categoría y `&& / ||` compilados como **saltos** —cortocircuito real, no un booleano calculado—. Verificado **ejecutando en la JVM propia**: `fib(10)=55` recursivo con rama, `fact(5)=120`, `break / continue`, aritmética `long / double`, y un objeto completo (`new M(10); m.add(5); m.get()` → 15). Tres bugs de corrección salieron de ahí: los constructores no emitían el `super()` implícito (§8.8.7, dejando el `this` sin inicializar), el `.class` no declaraba sus **campos** (referenciaba un `fieldref` inexistente) y faltaba el `Unary` de negación. También emite **excepciones**: `try / catch` con su tabla, `throw`, y el `finally` **duplicado** en la salida normal y en un handler *catch-all* que re-lanza (§14.20.2) —la v69 ya no acepta `jsr / ret`, así que duplicar es la única vía—. Eso, de paso, **reubicó** el *inlining* del `finally`, que teníamos anotado como tarea pendiente del *desugar*: no es una reescritura de AST sino trabajo del emisor. Y está la `StackMapTable` (§4.7.4), la pieza que separa «corre en mi JVM»

de «corre en cualquier JVM»: el frame de cada destino de salto se calcula como el **merge** de los estados que llegan ahí (un slot que no coincide en todos los caminos pasa a **Top**), siempre en forma **full_frame**, y los **handlers** llevan **stack = [E]** con el frame **forzado**, porque no los alcanza ningún salto. La validación es el **cross-check del verificador propio** contra su propia inferencia por punto fijo — que ya cazó un bug real: decidir «¿hace falta frame acá?» al fijar la etiqueta se saltaba el tope de los bucles, porque el **goto** hacia atrás se emite **después**. Y tiene **barrera de seguridad**: **generate()** devuelve **Result**, así que una construcción no soportada **falla con posición** en vez de emitir un **.class** a medias. Eso destapó al toque dos agujeros que el propio desugar venía alimentando en silencio — **instanceof** (todo **pattern switch**) y la creación de **arrays** (todo **varargs**)—, ya cerrados: el emisor cubre hoy **newarray / anewarray**, **arraylength**, las familias **Xaload / Xastore**, los inicializadores y el **instanceof**, de modo que un **pattern switch** y una llamada **varargs** **compilan y corren**. Y se cerró el resto del flujo de control: el **ternario** con sus dos ramas **promovidas al tipo del todo** (§15.25), el **switch** como **tableswitch** o **lookupswitch** según la **densidad** de sus etiquetas —con el relleno de alineación a 4 bytes y los desplazamientos de 4, preservando el **fall-through** de la forma de dos puntos y cortándolo en la de flecha—, los **break / continue** **con y sin etiqueta** sobre una pila de sentencias «de las que se puede salir» —un **switch** interpuesto captura un **break** pero **no un continue** (§14.16)— más el **bloque etiquetado**, y **synchronized** con **monitorexit** en **las dos salidas**, la normal y un handler **catch-all** que re-lanza. De yapa, el **switch sobre String** pasó a compilar de punta a punta: el desugar ya lo bajaba a dos switches sobre **int**, pero ninguno de los dos se podía emitir. Cerrar eso destapó el último agujero **silencioso** del emisor, y en la pieza más delicada: la pila de operandos se llevaba como un **simple contador de altura**, pero un frame del **StackMapTable** tiene que declarar **los tipos** de lo que hay en ella. Mientras todos los destinos de salto caían en bordes de sentencia —con la pila vacía— no se notaba; el ternario los mete en **medio de una expresión**: en **new M(a > 0 ? 1 : 2)** el **new** y su **dup** siguen vivos en la pila, y el frame los declaraba como pila vacía. El **.class** salía igual —la barrera no ve esto, porque el emisor cree estar emitiendo bien— y lo rechazaba recién el verificador. La pila pasó a llevarse **tipada** (48 puntos de ajuste), lo que de paso hizo representable el tipo **uninitialized** (tag 8), que identifica un objeto recién creado por el **offset de su new** y, corrido su **<init>**, pasa a definitivo en todas sus apariciones —pila y locales— (§4.10.2.4). **Después** se sumó el **invokedynamic** (opcode **0xba**): el pool ganó **MethodHandle / MethodType / InvokeDynamic** y el atributo **BootstrapMethods**, y un nodo **Indy** **genérico** (bootstrap + argumentos estáticos tipados) sirve tanto al **LambdaMetafactory** de las **lambdas/method refs** como al **ObjectMethods** de un **record** — con el emisor empujando las capturas y el verificador propio validando el call site. Y se cerró el frente de **clases internas, locales y anónimas**: la **captura del entorno** — **this\$0** de la instancia envolvente y **val\$x** de los locales **effectively-final**, con un pase **hoist_anonymous** que baja cada **new T(){...}** a una local sintética **Outer\$N** —, las tres formas **corren y verifican**. Por último, la **emisión de interfaces propias**: el **.class** de una **interface** sale con **ACC_INTERFACE** y sus métodos **abstractos sin Code**, y se sumó **invokeinterface** (con **InterfaceMethodref**) para las llamadas sobre un receptor de interfaz —lo que desbloqueó las **anónimas sobre una interfaz** (**new Runnable(){...}**), el caso más común y las lambdas contra interfaces funcionales propias—.

✓ **Siguiente (compilador)**: el **frente grande de invokedynamic quedó cerrado** —lambdas (con un **LambdaToMethod** propio), **referencias a método** (las cuatro formas del §15.13.1) y el **equals / hashCode / toString** de un **record** (por **ObjectMethods**) se tipan, emiten y **verifican**; la ejecución del call site vive en la JVM (KajijDK)—. Se cerraron además las **clases internas, locales y anónimas** —captura del entorno completa (**this\$0 + val\$**, con un pase **hoist_anonymous** que baja la anónima a una local sintética **Outer\$N**), las tres **corren y verifican**, incluidas las formas **cualificadas** (**Outer.this** → cadena de **this\$0**, con desambiguación de campo sombreado y multinivel; y **outer.new Inner()**, que empuja el calificador como **this\$0**) y los **argumentos de super** (**new ArrayList(10){...}**), la **colisión de nombres** (dos locales homónimas se renombbran a únicas **l1 / 2l** con alcance léxico por bloque), la **local-en-local** (anidada como tipo de su local envolvente) y el **multinivel implícito** (acceso a un miembro del abuelo vía **this\$0** encadenado) —las clases internas quedan **sin colas**—; el parser (B1) quedó **sin bordes** (anotaciones de tipo/uso **List<@NonNull String>**, **<@Foo T>**, **new <T>Foo()**, y las de **nivel de array** **String @A []**); y la **emisión de interfaces propias** cerró el caso más común de anónima (**new Runnable(){...}**) sumando **invokeinterface**. Y se cerró el **boxing dirigido por tipo** (**TransTypes**): una pasada propia inserta **Integer.valueOf / x.intValue** en cada contexto de conversión (asignación, **return**, argumentos, operandos, cuerpo de lambda), cerrando el caso insignia de las **poly expressions** — **Function<Integer,Integer> f = x -> x + 1** de punta a punta, validado por el verificador (ejecutar **valueOf** es de la JVM)—. Y se cerró la **inferencia en posición de argumento** (dos fases §15.12.2.6): un argumento **poly** —lambda, **method ref**, diamante— se re-atribuye con el tipo de su parámetro como **target** una vez resuelta la sobrecarga, lo que por fin baja una **lambda-argumento** (**call(x -> x + 1)**) a **invokedynamic**. Y se cerró el **enum con constantes parametrizadas** (**ROJO("rojo")**): al ctor propio se le antepone (**String, int**) + **super(...)** y cada constante se construye con sus argumentos. Y se ****liquidaron** los sueltos** que quedaban: **@Override / throws** en el chequeo de override (§8.4.8.3/§9.6.4.4), **RuntimeVisibleAnnotations** (§4.7.16), el **plegado de constantes** en un **case** (§15.28), las **formas comprimidas del StackMapTable** (§4.7.4), los **inicializadores de campo atribuidos** (§8.3.2), la **desambiguación de sobrecargas por el arg lambda** (§15.12.2.5) y el **boxing wrapper→primitivo más ancho** (**long l = anInteger**). Y se cerró el primer **frente grande de Java SE 25: sealed / non-sealed + exhaustividad de switch** (§8.1.1.2/§9.1.1.4/§8.1.6/§14.11.1.1) — el parser toma las keywords restringidas y el **permits**, el grafo gana los **subtipos autorizados** (con el **permits** implícito de la unidad), **Check** valida las reglas de sellado y exige **exhaustividad** a la **switch**-expresión y a la sentencia con **patterns** (jerarquía **sealed** recursiva, todas las constantes de un **enum, true / false** de un **boolean**; una guarda no cubre), y el **.class** lleva el atributo **PermittedSubclasses** (§4.7.31) — de yapa se corrigió que **case RED -> 1** se leía como lambda. Con eso la lista **tracked** del compilador queda vacía. Y se cerró el segundo frente de **Java SE 25**: los **text blocks** (§3.10.6) — el parser decodifica el **""...""** con el algoritmo del JLS (normalización de terminadores, descarte de la línea de apertura, **indentación incidental** mínima + blancos finales, y luego los escapes con **s** y la **continuación de línea <LF>**), dando un **String** común que se emite como cualquier literal. Y se cerró el tercer frente: la **capture conversion** (§5.1.10) con **variables de captura frescas** (un **RType::CAPTURE{id,upper,lower}** con cotas inline e **id** fresco para la **freshness**): se aplica en acceso a miembros y en argumentos, el subtipado gana **CAP <: upper** y **s <: CAP** por la cota inferior, y el caso insignia **swap(List<?>) → swapHelper(List<T>)** ahora **infiere T**. Y se cerró el cuarto frente: los **bridge methods** (§8.4.8.3/§15.12.4.5) — el emisor sintetiza los puentes (**ACC_BRIDGE + ACC_SYNTHETIC**) que hacen que la sobrescritura funcione en bytecode cuando la **erasure** difiere, por un **parámetro genérico borrado** (**Comparable<Foo> → compareTo(Object)**, **Node<Integer>.set(Integer) → set(Object)**) o por un **retorno covariante** (**A.get():Object / B.get():String**), reenviando al método real con **checkcast + invokevirtual**; verifica y **despacha** de punta a punta. Y se cerró el último frente grande: los **módulos** (§7.7/§4.7.25) — el parser toma el **module-info.java** con las cinco directivas (keywords restringidas), el emisor produce un **module-info.class** fiel (**ACC_MODULE**, atributo **Module** con **CONSTANT_Module / Package** y el **requires java.base mandated**), y **Check** valida la buena formación (directivas duplicadas, auto- **requires**); la resolución del grafo es de runtime (JPMS, track aparte). **Con esto, los cinco frentes grandes de Java SE 25 quedan cerrados del lado del compilador** —sealed + exhaustividad, text blocks, captura de wildcards, **bridge methods** y módulos (B5)—. Lo que le queda al compilador ****no**** son features de lenguaje sino ****fidelidad**** (B6). Ya se cerraron los cuatro primeros atributos: ✓ **Signature** (§4.7.9) —los genéricos ya ****no** se pierden** en el binario—, ✓ **InnerClasses** (§4.7.6), ✓ **EnclosingMethod** (§4.7.7) —anidamiento y método envolvente de local/anónimas— ✓ **Record** (§4.7.30) —un **record** ahora extiende **java.lang.Record**, es **final** y lleva el atributo con sus componentes, así **isRecord() / getRecordComponents()** andan (y se cerraron dos bugs latentes: el parseo de un record genérico y la erasure del descriptor de una variable de tipo)— y ✓ **NestHost / NestMembers** (§4.7.28/§4.7.29) —el **nest** plano (**host + nestmates** transitivos) que da acceso privado entre anidadas—, ✓ **NestHost / NestMembers** (el **nest**) y ✓ **MethodParameters** (§4.7.24, los nombres de parámetros para la reflexión). De la ****inferencia**** ya se cerraron el lado escritura de la captura por llamada y el **lub con intersección** (**Number & Comparable**); queda solo el **solver** transitivo de §18 (bounds cruzados). Del lado de los **atributos ya no queda fidelidad reflexiva pendiente**: se cerraron **Exceptions**, **LineNumberTable**, **LocalVariableTable**, **ConstantValue** y **todas** las type annotations (**RuntimeVisibleTypeAnnotations**, §4.7.20 — parámetros de tipo, cotas, extends/implements, throws, field/return/param y los targets de **Code**: **cast / instanceof / new / local**), todo verificado byte-fiel contra el **javap** de Temurin.

Fase H — KajiLibrary (la biblioteca, dogfooded). La biblioteca propia vive en `KajiLibrary/`, se compila con el `javac` propio y corre en KajiJDK. **H1** cerró los *tiers* 0–5 (el esqueleto: `Object / String / StringBuilder`, excepciones, interfaces funcionales, colecciones); **H2** (utilidades y conveniencia de `java.util`) está **en curso**; **H3** (`java.util.stream`), **H4** (`java.time`) y **H5** (`java.util.regex`) tienen su **núcleo cerrado**, y **H6** (`java.util.Formatter / String.format`) está **diseñado**. Lo acoplado al runtime — `java.nio`, `java.util.concurrent` y la reflexión completa— queda en el backlog (**H7**): esa parte avanza por el track de la JVM, no por el de la biblioteca.

✓ **Siguiente:** con **H4 (JMM relajado)**, **H5 (CAS — `compareAndSwap` + los `Atomic*`)** y el **núcleo + primeras utilidades de H6 (AQS exclusivo+compartido → `ReentrantLock / Condition`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `ArrayBlockingQueue` / `LinkedBlockingQueue` / `PriorityBlockingQueue` / `DelayQueue`, el framework `Executor` (`ThreadPoolExecutor` + `ScheduledThreadPoolExecutor`), `ConcurrentHashMap` y `CompletableFuture`)** ya hechos y validados por el oráculo, sigue el **volumen restante de `java.util.concurrent`** (deques, *resize*/lecturas lock-free del `ConcurrentHashMap`) y los **locks finos por estructura** (ver la sección «Concurrencia»). Los green threads quedan como substrato elegible y **oráculo determinista** de corrección (`green=0s-gil=0s`) — clave para cazar el **bug residual** del modo paralelo (referencia *stale* bajo GC intenso; guard parcial landeado, próximo paso `ThreadSanitizer`). El paralelismo de cómputo del LLM vive en **kernels off-heap** (rayon/CUDA).